

Learn Python

学生版（中文）

十次两小时课程 + 毕业项目 · 可运行示例 · 课后作业 · 陷阱速查表 · 自测题

为零编程基础的研究者定制的
快节奏自学 Python 课程。

交互式网站的离线伴读版。
生成于 2026-07-07 · 教学内容均为原创

Learn Python —— 学生课程大纲

欢迎。这是一条约 20 小时的快速路径（十次两小时课程，外加可选的毕业项目），从"从没写过代码"到"我能写出真正的 Python 程序来处理我的研究数据"。课程顺序为你重新编排，并把最容易绊倒人的语言怪癖放在最前面。

每次 2 小时的课怎么上：先用 5 分钟热身，回顾上次的陷阱和作业；然后是主题的 **核心**部分（讲概念 → 我写你看 → **你来写**）；短暂休息后进入**更进一步**——围绕同一主题、货真价实的第二小时新内容，配自己的现场示例和练习；最后以回顾 + 测验 收尾。每个示例你都要亲手敲一遍。

课后作业：每课布置约 30–45 分钟作业（该课 `practice.md` 的 **Homework** 部分）。作业不计入课堂时间，附参考答案，下一课的热身从它开始。每个练习文件里的 **In class — going deeper** 部分属于该课的第二小时。

你需要准备：Python 3.11+、VS Code（或任何编辑器）和这个文件夹。每课打开对应的 `slides/`、`examples/`、`cheatsheets/` 文件。（整套课程可从网站下载 **PDF**，每课同时 也是可在浏览器中运行的 **Jupyter 笔记本**——无需安装。）

十次课（每次 2 小时，另有作业）

#	标题	你将学会.....
1	运行 Python、变量与类型	运行代码（REPL + 脚本），使用 <code>int/float/str/bool/None</code> 、f-string、类型转换，读懂报错
2	动态类型陷阱	分清 <code>==</code> 与 <code>is</code> ，预测 <code>True==1 / 0.1+0.2 / 5=="5"</code> ，用 <code>isinstance</code> 查类型，真值判断
3	控制流：条件与循环	<code>if/elif/else</code> 、链式比较、 <code>for/while</code> 、 <code>break/continue</code> 、 <code>enumerate/zip</code> 、输入校验循环
4	数据结构	<code>list/tuple/dict/set</code> 、切片、"字典列表即数据集"、推导式、排序、别名
5	函数、作用域与复用	写可复用的函数、 <code>*args/**kwargs</code> 、类型标注、LEGB、躲开可变默认值 bug
6	递归与递归思维	基例 + 递归步、追踪调用栈、在嵌套数据上递归、知道它的极限
7	异常与防御式编程	用 <code>try/except</code> 、 <code>raise</code> 、EAFP 校验脏输入，写第一个 <code>pytest</code> 测试
8	文件、库与研究数据	读写 CSV 问卷数据、 <code>statistics/datetime/pathlib</code> 、 <code>pip install</code> 、 <code>pandas</code> 预览
9	正则表达式与文本清洗	用 <code>re</code> + 捕获组校验、提取、清洗真实文本
10	模块、面向对象与 Python 惯用法	导入模块、写带 <code>@property</code> 的小类、生成器/ <code>map/filter</code> /海象运算符
11	毕业项目（可选）	端到端搭一个"成绩册与问卷分析器"

第 N 课的文件：`slides/session-NN-slides.md` · `examples/session-NN/`。第 2 课的类型陷阱是全程承重最大的内容——如果只精通一课，就选它。

常备工具箱（随时打开）

- `cheatsheets/traps-and-gotchas.md` —— 每个怪癖的错误写法 vs 正确写法。一直开着。
- `cheatsheets/quick-reference.md` —— 总记不住的语法（切片、f-string、推导式）。
- `cheatsheets/glossary.md` —— 每个术语的大白话释义。

学习方法（针对这套材料）

1. **动手敲，别光看。** 把 `examples/` 里的示例重新敲一遍；改动一处，先预测结果再运行。
2. **对陷阱先预测再运行。** 尤其第 2 课：先猜输出，再运行。惊讶就是课程本身。
3. **作业当天做。** 趁热打铁 30–45 分钟；真正尝试过之后再看答案。
4. **记一本“我踩过的坑”。** 每次报错，抄下最后一行 + 你的修法。你会攒出自己的专属速查表（第 2 课的作业就是它的开头）。
5. **联系你已有的知识。** 每个主题都对应研究方法/统计的概念（见 `connection-map.md`）。多用这些桥。

什么算“学完了”

完成毕业项目（第 11 课），或至少做完每课的练习 + 作业、通过 `assessments/` 里的每课测验。真正的检验是：你能打开一份自己的脏 CSV，独立写出一个汇总它的小脚本。

范围（免得你意外）

这门课让你成为一名能处理研究数据的程序员。它不是完整的数据科学课——pandas、绘图、统计建模只浅尝辄止。等这些根基牢固后，那是自然的下一步。

第 1 课

运行 Python、变量与类型

为什么是 Python（对你而言）

- 免费、易读，是研究计算领域的通用语言。
- 是连接你的数据、统计与写作的"胶水"。
- 今天的目标：从零到"我跑通了一个程序"。

贯穿全课程的座右铭：**可读性至上**。

运行 Python 的两种方式

1. **REPL（交互式）** —— 输入 `python3`，出现 `>>>`，一次运行一行。适合做实验。
2. **脚本** —— 保存 `hello.py`，运行 `python3 hello.py`。适合正式工作。

```
# hello.py
print("Hello, researcher")
```

变量 = 贴在对象上的标签

```
n = 30          # 一个 int（整数）
mean = 3.7      # 一个 float（浮点数）
name = "Ada"    # 一个 str（字符串）
passed = True   # 一个 bool（布尔值）
missing = None  # "没有值"
```

= 是一个**动作**（"把这个标签贴到那个对象上"），**不是**数学等式。所以 `n = n + 1` 完全合法。

□ 统计里你也给量命名（`n`、`α`）——思路相同，只是这里的名字可以随时改贴。

五大核心类型

类型	示例	联想
int	30	计数
float	3.7	测量值
str	"Ada"	文本
bool	True / False	开关标志
NoneType	None	缺失

type(x) 会告诉你 x 是哪种类型。

输入与输出

```
name = input("Your name: ")      # △ 永远是 str
age  = int(input("Your age: "))   # 立即转换
print("Hi", name, "- age", age)
```

第一周的头号陷阱：input() 给你的是文本。 "5" + "3" 是 "53"，不是 8。

f-string（请用这个）

```
score = 87.456
print(f"{name} scored {score:.1f}")  # 保留一位小数
print(f"{1234567:,}")               # 1,234,567
print(f"{0.873:.1%}")               # 87.3%
```

比用 + 拼接干净得多，也不会有类型错误。

类型转换

```
int("42")      # 42
float("3.14")  # 3.14
str(42)        # "42"
int(3.9)       # 3   （截断，不是四舍五入！）
round(3.9)     # 4
```

int(input(...)) = 读入文本、转成数字，一步完成。

读懂报错信息（别慌）

```
Traceback (most recent call last):
  File "x.py", line 3, in <module>
    age = int(input("Age: "))
ValueError: invalid literal for int() with base 10: 'thirty'
```

先读最后一行。它说明了问题类型（`ValueError`）和出错的值（`'thirty'`）。

现场演示 & 轮到你了

- 演示：先写 `greet.py`，再做一个“距离毕业还有几年”的计算器。我们会故意弄坏一次。
- 你来：`examples/session-01/practice.md` —— 写一个 GPA/BMI 风格的小脚本。

更进一步

数字、字符串，以及它们周边的工具

算术运算，完整版

运算符	含义	示例
<code>+ - *</code>	常规运算	<code>3 * 7 → 21</code>
<code>/</code>	真除法	<code>7 / 2 → 3.5</code>
<code>//</code>	整除（向下取整）	<code>7 // 2 → 3</code>
<code>%</code>	取余	<code>7 % 2 → 1</code>
<code>**</code>	乘方	<code>2 ** 10 → 1024</code>

优先级与数学一致（先 `**`，再 `* / // %`，最后 `+ -`）——拿不准时，**加括号**。就地更新：`score += 5`、`count -= 1`、`total *= 2`。

% 的用武之地

```
n % 2 == 0          # 偶数？
student_id % 3      # 0、1 或 2 -> 轮流分进 3 个讨论组
minutes = 130
print(minutes // 60, "h", minutes % 60, "min")  # 2 h 10 min
```

□ `//` 和 `%` 搭配，把一个量拆成“整份 + 余数”——小时/分钟、页数/张数、分组/余下。

字符串自带方法

```
name = " aDA lovelace "  
name.strip()          # "aDA lovelace"    ( 去掉首尾空白 )  
name.strip().title()  # "Ada Lovelace"    ( 可以链式调用 ! )  
"CS50".lower()       # "cs50"  
"a,b,c".count(",")   # 2  
"ana@uni.edu".startswith("ana") # True  
"@ " in "ana@uni.edu" # True    ( 成员判断 )  
len("data")          # 4
```

方法**返回新字符串**——原来的从不变（字符串不可变）。字符串还能“乘”：`"ab" * 3` → `"ababab"`
——`print("=" * 40)` 画一条分隔线。

f-string 进阶——对齐的报表

```
name, score, rate = "Ana", 91.456, 0.873  
print(f"{name:<10}{score:>8.1f}{rate:>8.1%}")  
# Ana          91.5   87.3%
```

- `<10` 左对齐补到 10 位 · `>8` 右对齐到 8 位
- `.1f` 一位小数 · `,` 千位分隔 · `.1%` 百分比
- `{score=}` 会打印 `score=91.456` ——调试神器。

print() , 微调版

```
print("Ana", "Ben", "Cara", sep=" | ") # Ana | Ben | Cara  
print("Loading", end="...")           # 不换行——下一个 print 接着打  
print('She said "wow"')               # 换一种引号.....  
print("She said \"wow\"")              # .....或者用反斜杠转义  
print("line one\nline two")            # \n = 字符串里的换行符
```

- `sep=` 决定参数之间的连接符（默认空格）；`end=` 决定行尾（默认 `"\n"`）。
- `\n` 和 `\"` 是**转义序列**——戴着反斜杠帽子的单个字符。

借用工具箱：import

```
import math  
math.sqrt(144) # 12.0  
math.floor(3.9) # 3  
math.ceil(3.1) # 4  
math.pi        # 3.141592653589793
```

一行代码借来一整个库。（`import` 的完整故事——连同 `pip` ——在第 8 课。）

直接问 Python 本人

```
help(round)      # 单个函数的说明书
dir(str)         # 字符串的全部方法
```

在 REPL 里，`_` 保存着上一个结果。这三个习惯能替你省下一半的网络搜索。

起不咬人的名字

- 变量和函数用 `snake_case`（蛇形命名）：`class_average`，而不是 `ClassAvg`。
- 名字要说明它是什么：`n_students`，而不是 `x`。
- 常量的约定写法：`MAX_SCORE = 100`（Python 不会强制——全大写是提醒人类的）。
- 注释解释为什么，而不是“是什么”：代码本身已经说明了是什么。
- 动手之前，先用大白话把步骤写出来（伪代码）——然后再翻译成代码。

轮到你了——第二轮

`examples/session-01/practice.md` → **In class — going deeper**：格式化练习、姓名清洗器，以及 % 分组练习。

陷阱回顾

- `input()` → 永远是字符串；用 `int()` / `float()` 转换。
- `print(a, b)`（逗号 → 空格）vs `print(a + b)`（必须同类型）。
- `int(3.9)` 截断成 `3`；要四舍五入用 `round()`。

小结

你已经会运行代码、给值命名、转换类型、格式化输出、读懂报错。下一课：第 2 课——动态类型陷阱。

课后作业（第 2 课之前）

课外完成——不计入课堂时间。完整题目 + 参考答案：`examples/session-01/practice.md` → *Homework*。

1. **单位换算器** —— 一个脚本：读入数值，用 f-string 漂亮地输出换算结果。
2. **报错演练** —— 故意弄坏三行代码；逐个读最后一行，说出错误名称。
3. **类型预测表** —— 先预测十个表达式的 `type(...)`，再到 REPL 里验证。

练习与作业

课堂练习

任务 1 —— REPL 热身

在 REPL (`python3`) 里查看这些的类型： `42`、`42.0`、`"42"`、`True`、`None`、`7/2`、`7//2`。哪个让你意外？（提示：`7/2`。）

任务 2 —— GPA 报告器 (`gpa.py`)

读入姓名和 GPA (小数)。打印：`"<name>'s GPA is <保留两位小数>, which is <87.5%> of a 4.0 scale."`

任务 3 —— 问卷年龄段 (`age.py`)

读入年龄 (整数)。打印年龄，以及“活过的月份数” (年龄 $\times$ 12)。然后故意输入 `twenty` 而不是数字，读一读报错的最后一行。

任务 4 —— 进阶：字符串陷阱

不做转换时，输入 `2` 和 `3`，`input("a: ") + input("b: ")` 会打印什么？现在修好它，让它打印 `5`。

加餐 —— Python 惯用法速练

盖住 `# ->` 后面的答案，逐行预测，再运行。

```
a, b = 1, 2
a, b = b, a;          print(a, b)          # -> 2 1    ( 交换, 不用临时变量 )
print(f"{a + b = }")   # -> a + b = 3    ( 自带说明的 f-string )
print("CS50".lower(), " hi ".strip())      # -> cs50 hi
print("@" in "ana@uni.edu", len("data"))    # -> True 4    ( 成员判断 + 长度 )
```

课堂练习——更进一步 (第二小时)

E1 —— f-string 格式化速练

给定 `pi = 3.14159265`、`n = 9876543`、`rate = 0.4567`，精确打印出：`3.14 · 9,876,543 · 45.7% · pi = 3.14159265` (自带说明的形式)。

E2 —— 入学周数

扩展 `age.py`：再报告年龄折合的周数 (每年 52 周)，带千位分隔符。

D1 —— 对齐的成绩单行

把三名学生 (`name`, `score`, `rate`) 打印成对齐的列：姓名左对齐补到 10 位、 分数右对齐保留 1 位小数、 比率用百分数。三行必须对得整整齐齐。

D2 —— 姓名清洗器

用链式字符串方法把 " aDA lovelace " 变成 "Ada Lovelace" , 然后报告结果的 `len()` , 以及它是否 `.startswith("Ada")` 。

D3 —— 整份 + 余数

1. 用 `//` 和 `%` 把 130 分钟换算成 2 h 10 min。
2. 用 `%` 把学号 [101, 102, 103, 104, 105, 106] 轮流分进 0/1/2 三个讨论组。

课后作业（第 2 课之前）

约 30–45 分钟，课外完成——不计入课堂时间。先都试一遍，再看答案。

H1 —— 单位换算器 (`convert.py`)

读入英里数 (可以是小数) 。打印公里数 (`× 1.60934` , 保留 2 位小数) 和带千位 分隔符的米数： `5.0 miles = 8.05 km (8,047 meters)`

H2 —— 报错演练

在脚本里逐个故意写坏下面三行，运行，并抄下每个报错的最后一行：

1. `int("ten")`
2. `"age: " + 21`
3. 在 `name = "Ada"` 之后写 `print(nmae)` (故意打错) 然后各用一句话说明：错误的名字告诉你什么？

H3 —— 类型预测表

先预测每个的 `type(...)` (或输出) , 再到 REPL 里验证： `7/2 · 7//2 · 7.0//2 · "7"*2 · int("7")*2 · 7 == 7.0 · None · input ( 不加括号 ! ) · print("hi")`

参考答案

课堂练习

```
# 任务 2 — gpa.py
name = input("Name: ")
gpa = float(input("GPA: "))
print(f"{name}'s GPA is {gpa:.2f}, which is {gpa/4:.1%} of a 4.0 scale.")

# 任务 3 — age.py
age = int(input("Age: "))
print(f"You are {age} years old, about {age*12} months.")
# 输入 "twenty" -> ValueError: invalid literal for int() with base 10: 'twenty'

# 任务 4
# "2" + "3" -> "23" (字符串拼接)
a = int(input("a: ")); b = int(input("b: "))
print(a + b)          # 5
```

课堂练习——更进一步

```
pi, n, rate = 3.14159265, 9876543, 0.4567
print(f"{pi:.2f}")          # 3.14
print(f"{n:,}")             # 9,876,543
print(f"{rate:.1%}")        # 45.7%
print(f"{pi = }")           # pi = 3.14159265

# E2
age = int(input("Age: "))
print(f"That's about {age * 52:,} weeks.")

# D1
for name, score, rate in [("Ana", 91.456, 0.873), ("Ben", 58.0, 0.412), ("Cara", 73.2, 0.65)]:
    print(f"{name:<10}{score:>8.1f}{rate:>8.1%}")

# D2
clean = " aDA lovelace ".strip().title()
print(clean, len(clean), clean.startswith("Ada")) # Ada Lovelace 12 True

# D3
minutes = 130
print(f"{minutes // 60} h {minutes % 60} min")      # 2 h 10 min
for sid in [101, 102, 103, 104, 105, 106]:
    print(sid, "-> group", sid % 3)
```

课后作业

```
# H1 — convert.py
miles = float(input("Miles: "))
km = miles * 1.60934
print(f"{miles} miles = {km:.2f} km ({round(km * 1000):,} meters)")
```

H2 —— 最后一行及其含义：

1. `ValueError: invalid literal for int() with base 10: 'ten'` —— 参数类型对，值不能用。
2. `TypeError: can only concatenate str (not "int") to str` —— 类型本身与操作不匹配。
3. `NameError: name 'nmae' is not defined` —— 用了从未赋值过的名字（通常是笔误）。

```
# H3 — type() 会说什么
7 / 2          # float — / 永远给浮点数 (3.5)
7 // 2         # int  — 两个整数的整除 (3)
7.0 // 2       # float — 值取整，类型是浮点 (3.0)
"7" * 2        # str  — 重复："77"，不是数学
int("7") * 2   # int  — 14
7 == 7.0       # bool — True (数字按值比较)
None           # NoneType
input          # builtin_function_or_method — 函数也是值
print("hi")    # 打印 hi，然后求值为 None (print 不返回东西)
```

陷阱 —— 先预测，再核对

盖住结果，逐个预测，再自己核对。

1.

```
score = '5'
bonus = '3'    # what input() would hand you
score + bonus
```

结果: `'53'`。这两个是字符串（`input()` 给的永远是字符串）：`+` 拼接文本。先转换：`int(score) + int(bonus)`。

2.

```
age = 21
'Age: ' + age
```

结果: `TypeError: can only concatenate str (not "int") to str`。文本和数字不能用 `+` 粘在一起。用 f-string——`f"Age: {age}"`——或 `print('Age:', age)`。

3.

```
gpa = 3.9
int(gpa)
```

结果: `3`。`int()` 向零截断，不做四舍五入。想四舍五入用 `round(gpa)`。

4.

```
name = 'ada'
name.upper()
name
```

结果: `'ada'`。字符串不可变：`.upper()` 返回新字符串，原来的不动。要赋回去：`name = name.upper()`。

第 2 课

动态类型陷阱

名字只是标签

Python 不会把名字锁定在某个类型上——它现在可以指向 `int`，过会儿又指向 `str`。这种自由很强大，但也造出了陷阱：结果会和你的直觉相反。

本课全程动手。下面每条规则在页面底部的 **陷阱——先预测，再运行** 里都有一行可运行的对应代码。先预测，再揭晓。

`==` 比值，`is` 比身份

- `==` 问“值相同吗？”——几乎永远是你想要的。
- `is` 问“是同一个对象吗？”——只用于 `None`（以及 `True / False`）。
- `b = a` 不会复制：两个名字指向同一个列表，改 `a` 会“透过”`b` 显现。复制要用 `list(a)` 或 `a[:]`。

□ GPA 相同 = `==`。是同一个学生 = `is`。

布尔值就是整数

- `bool` 是 `int` 的子类型，所以 `True` 表现为 `1`，`False` 表现为 `0`。
- 这意味着 `5 + True` 能算，`sum(flags)` 能数出有多少个 `True`。
- 但身份仍然不同：`isinstance(True, int)` 为真，而 `type(True) is int` 为假。

□ 相当于语言里内置了哑变量编码（1/0）。

数字：int、float、除法

- `/` 永远返回浮点数（`4 / 2` 是 `2.0`）；`//` 向负无穷取整。
- `3 == 3.0` 按值比较相等，但小数以二进制存储，所以 `0.1 + 0.2` 并不精确等于 `0.3`。
- 永远不要用 `==` 比较两个浮点数——用 `math.isclose(a, b)`。（你本来也从不对两个测量分数做精确相等判断，直觉是一样的。）

跨类型比较

- 跨类型的 `==` / `!=` 返回 `False`，从不报错。
- 跨不兼容类型的 `<` / `>` 会抛出 `TypeError`——计算机能回答"相同吗？"，却无法给文本和数字排大小。
- 内容相同的列表和元组**永远不相等**；序列是逐元素比较的。

真值判断 (truthiness)

- 假值：`0` `0.0` `""` `[]` `{}` `set()` `None`。真值：其余一切——包括 `"0"`、`"False"` 和 `[0]`。
- 惯用写法：`if scores:` 而不是 `if len(scores) > 0:`。
- 但用户输入的文本要先转换——`"0"` 是真值。

更进一步

None、类型转换与精确小数

None，讲透它

- `None` 表示**缺失**——"这里没有值"——不是零，也不是空。
- 用身份判断：`if x is None:`（永远不要写 `== None`）。
- 没有 `return` 的函数返回的就是 `None`（第 5 课还会遇到它）。
- 三种不同的"没有"：`None`（缺失）· `""`（空文本）· `0`（真实的零）。问卷里的空格是 `None` 性质的；得 0 分的学生不是缺失。

类型转换矩阵

调用	结果
<code>int("42")</code>	<code>42</code>
<code>int("4.2")</code>	<code>ValueError</code>
<code>float("4.2")</code>	<code>4.2</code>
<code>int(4.9)</code>	<code>4</code> （截断）
<code>int(float("4.2"))</code>	<code>4</code> —— 两步走
<code>str(4.2)</code>	<code>"4.2"</code> （任何东西都能转成 <code>str</code> ）

规则：在入口处转换，只转一次——数据一进来，就把类型定对。

nan 与 inf

```
bad = float("nan")      # "not a number" — 失败/缺失的数值
bad == bad               # False! NaN 不等于任何东西
import math
math.isnan(bad)          # True — 唯一可靠的检测
float("inf") > 10**100    # True — 无穷大胜过一切
```

NaN 会出现在真实的数据管道里（pandas 用它表示缺失单元格）——现在就认识它。

可变 vs 不可变

不可变（改不了）	可变（可以改）
<code>int, float, bool, str, tuple, None</code>	<code>list, dict, set</code>

```
id(x)    # 对象的身份编号 — `is` 实际比较的就是它
```

不可变对象可以放心共享；可变对象会产生别名（`b = a` 是共享！）。这正是第 4 课的头号陷阱——现在你知道了为什么。

浮点数不够用时：Decimal

```
from decimal import Decimal
Decimal("0.1") + Decimal("0.2") == Decimal("0.3")    # True!
```

- 精确的十进制运算——要从字符串构造，不要从浮点数。
- 用于金钱，以及任何 `isclose` 不能接受的记账场合。
- 代价：更啰嗦、更慢——日常默认仍然是浮点数。
- （还有 `fractions.Fraction`：精确的分数，如 `Fraction(1, 3)`。）

链式比较，再看一眼

```
1 <= likert <= 5        # 两端都检查 — 读起来像数学
a == b == c              # 三者全相等
```

但不要链 `is`，也不要耍小聪明混用运算符——只在读起来恰好像你想表达的数学时才用链式。

轮到你了——第二轮

`examples/session-02/practice.md` → **In class — going deeper**：类型转换预测表、用 `Decimal` 重算 `0.1 + 0.2`，以及能区分 `None` / `""` / `0` 的 `describe(x)`。

现在，踩响这些陷阱

打开下方的 **陷阱** —— **先预测，再运行**：约 18 个单行陷阱。对每一个，先大声说出你的预测，再运行，看真实结果和背后的原因。

然后在 `examples/session-02/practice.md` 里：写 `clean_score(value)`，安全地接受 `5`、`5.0` 或 `"5"` 并返回浮点数，拒绝一切无意义输入。

下一课：第 3 课——控制流：条件与循环。

课后作业（第 3 课之前）

课外完成——不计入课堂时间。完整题目 + 参考答案：`examples/session-02/practice.md` → *Homework*。

1. **陷阱日志** —— 挑出最让你意外的 5 个陷阱，各用一句自己的话解释原因。
2. `approx_equal(a, b)` —— 从今往后替代 `==` 的浮点数比较函数。
3. `is_missing(x)` —— 正确地把 `None` 和 `0`、`""`、`False` 区分开。

练习与作业

课堂练习

预测输出闯关

对每一行，写出为什么（一句话）。先预测，再运行验证。

```
True + True           # 2      — 布尔值是整数；True 是 1
3 == 3.0              # True   — 数字按值比较
0.1 + 0.2 == 0.3      # False  — 二进制浮点舍入
5 == "5"              # False  — 类型不同，不报错
5 > "5"               # □     — int 和 str 无法排大小
[1,2] == (1,2)        # False  — 列表和元组是不同类型
bool("0")              # True   — 非空字符串是真值
x=[1]; y=x; x.append(2); y # [1,2] — y 是 x 的别名
```

编写 `clean_score()`

写一个函数，把值安全地转成 0–100 范围内的浮点数：

```
def clean_score(value):
    """
    接受 87、87.0 或 "87", 返回 87.0 (浮点数)。
    - 超出 0..100 的值给出明确提示并拒绝 (返回 None)。
    - 浮点比较要安全 (不用精确 ==)。
    """
```

用这些测试：87、87.0、"87"、"eighty"、120、True。True 会怎样？为什么？（提示：bool 是一种 int.....）

加餐 —— Python 惯用法速练

盖住 # -> 后面的答案，逐行预测，再运行。

```
x = int("257"); y = int("257")
print(x == y, x is y)           # -> True False    (值相等，对象不同)
print(float("nan") == float("nan")) # -> False    (NaN 不等于任何东西，包括它自己)
```

课堂练习——更进一步（第二小时）

E1 —— isinstance 速练

逐个预测，再运行：

```
isinstance(True, int)
type(True) is int
isinstance(3.0, int)
isinstance("3", (int, float))
isinstance(3, (int, float))
```

E2 —— 值相同，对象相同吗？

不运行：执行 `a = [1, 2]; b = [1, 2]; c = a` 之后，`a == b`、`a is b`、`a is c` 各是什么？接着 `c.append(3)` —— `a` 变成什么？运行验证全部四问。

D1 —— 类型转换预测表

逐个预测，再运行：`int("42") · int("4.2") · float("4.2") · int(4.9) · int(float("4.2")) · str(4.2) + "!" · bool("False")`。

D2 —— Decimal 重算

证明 `Decimal("0.1") + Decimal("0.2") == Decimal("0.3")` 而浮点版为 False——再检查成绩权重：`0.1 + 0.2 + 0.3 == 0.6` 用 `Decimal` 能修好吗？为什么 `Decimal` 必须从字符串构造？

D3 —— `describe(x)`

`None` 返回 `"missing"`、`""` 返回 `"empty"`、`0/0.0` 返回 `"zero"`（但 `False` 不算），其余返回 `"value"`。在 `[None, "", 0, 0.0, False, "0", 5]` 上验证。（小心：`False == 0` —— 正是第 2 课自己的陷阱。哪个工具能区分它们？）

课后作业（第 3 课之前）

约 30–45 分钟，课外完成——不计入课堂时间。先都试一遍，再看答案。

H1 —— 陷阱日志

从今天约 18 个陷阱中挑出**最让你意外的五个**。每个写下：那行代码、你原本的 错误预期、以及 Python 为什么那样回答——一句话，用自己的话。（这本日志是你 专属速查表的种子。）

H2 —— `approx_equal(a, b, tol=1e-9)`

写出你从今往后替代 `==` 的浮点比较；它必须让 `approx_equal(0.1 + 0.2, 0.3)` 得到 `True`。手写一版（`abs`），再用 `math.isclose` 写一版。

H3 —— `is_missing(x)`

只对 `None` 返回 `True`——对 `0`、`0.0`、`""`、`False` 都不行。在全部五个值 上验证。（提示：这正是 `is` 的用武之地。）

参考答案

课堂练习

```
import math

def clean_score(value):
    # 显式拒绝 bool — 否则它会冒充 int 混进来 (True == 1)。
    if isinstance(value, bool):
        print(f"Rejected {value!r}: looks like a flag, not a score.")
        return None
    try:
        score = float(value)          # 同时处理 int、float 和数字字符串
    except (ValueError, TypeError):
        print(f"Rejected {value!r}: not a number.")
        return None
    if not 0 <= score <= 100:
        print(f"Rejected {value!r}: out of range 0-100.")
        return None
    return score

for v in [87, 87.0, "87", "eighty", 120, True]:
    print(v, "->", clean_score(v))
# 87->87.0, 87.0->87.0, "87"->87.0, "eighty"->None, 120->None, True->None
```

关键教训：`float(True)` 是 `1.0`，不显式检查 `bool` 的话，一个开关标志就会冒充有效分数。这就是 `bool < int` 陷阱进了真实函数的样子。

课堂练习——更进一步

```
# E1
isinstance(True, int)          # True — bool 是 int 的子类
type(True) is int             # False — 精确类型是 bool
isinstance(3.0, int)          # False — float 不是 int 的子类
isinstance("3", (int, float)) # False — 长得像数字的字符串仍是 str
isinstance(3, (int, float))    # True — 元组表示"其中任何一种"

# E2
a == b    # True — 值相同
a is b    # False — 两个不同的列表对象
a is c    # True — c 是 a 的对象上的另一个标签
# c.append(3) 之后: a == [1, 2, 3] — 通过 c 的修改透过 a 显现

# D1
int("42")          # 42
# int("4.2")        -> ValueError — int() 只解析整数文本
float("4.2")        # 4.2
int(4.9)           # 4 (截断)
int(float("4.2"))   # 4 (两步走)
str(4.2) + "!"     # '4.2!'
bool("False")      # True — 任何非空字符串都是真值！

# D2
from decimal import Decimal
print(Decimal("0.1") + Decimal("0.2") == Decimal("0.3")) # True
print(Decimal("0.1") + Decimal("0.2") + Decimal("0.3") == Decimal("0.6")) # True
# 要从字符串构造: Decimal(0.1) 会把浮点数的二进制误差原样带进来。

# D3
def describe(x):
    if x is None:
        return "missing"
    if x == "":
        return "empty"
    if isinstance(x, bool):          # bool < int — 先查开关标志
        return "value"
    if isinstance(x, (int, float)) and x == 0:
        return "zero"
    return "value"

for v in [None, "", 0, 0.0, False, "0", 5]:
    print(repr(v), "->", describe(v))
# None missing · "" empty · 0 zero · 0.0 zero · False value · "0" value · 5 value
```

课后作业

H1 —— 任选五个，例如：`0.1 + 0.2 == 0.3`（二进制浮点）、`True == 1`（`bool < int`）、`5 == "5"`（不做跨类型转换）、`[1, 2] == (1, 2)`（列表 ≠ 元组）、对相等列表用 `is`（同一 ≠ 相等）。用自己的话写为什么比挑哪个更重要。

```
# H2
import math

def approx_equal(a, b, tol=1e-9):
    return abs(a - b) <= tol
    # 或者: return math.isclose(a, b, abs_tol=tol)

print(approx_equal(0.1 + 0.2, 0.3))    # True

# H3
def is_missing(x):
    return x is None                  # 身份判断 — 只有 None 本身能通过

for v in [None, 0, 0.0, "", False]:
    print(repr(v), "->", is_missing(v))    # 只有 None -> True
```

陷阱 —— 先预测，再核对

盖住结果，逐个预测，再自己核对。

1.

```
ana_scores = [88, 91]
ben_scores = [88, 91]
ana_scores is ben_scores
```

结果: `False`. `==` 比较值 (`ana_scores == ben_scores` 为 `True`) ; `is` 比较身份——是否是内存中的同一个对象。Ana 和 Ben 的列表只是恰好相同。比值用 `==` , `is` 留给 `None` 。

2.

```
roster = ['Ana', 'Ben']
backup = roster
roster.append('Cara')
backup
```

结果: `['Ana', 'Ben', 'Cara']`. `backup = roster` 不会复制：两个名字指向同一个列表，通过 `roster` 追加也会在 `backup` 里出现。复制用 `list(roster)` 或 `roster[:]` 。

3.

```
attended = True
attended == 1
```

结果: `True`. `bool` 是 `int` 的子类: `True == 1`、`False == 0`——这正是电子表格里的 1/0 与 Python 的 `True/False` 对得上的原因。

4.

```
score = 5
bonus = True    # earned a bonus point?
score + bonus
```

结果: 6. 算术中 `True` 相当于 1 (`False` 相当于 0) ——给标志加一分的快捷方式。

5.

```
passed = [True, False, True, True]    # did each student pass?
sum(passed)
```

结果: 3. 对布尔值求和就是数 `True` 的个数——统计多少学生通过的顺手办法。

6.

```
homework = 0.1
exam = 0.2    # their weights in the final grade
homework + exam
```

结果: 0.30000000000000004. 浮点数按二进制存储, 0.1 和 0.2 都没有精确表示, 小误差累加了。

7.

```
weight = 0.1 + 0.2    # homework + exam
weight == 0.3
```

结果: False. 因为 0.1 + 0.2 是 0.30000000000000004。永远别用 == 检验计算出的分数; 用 `math.isclose(a, b)`。

8.

```
missing_score = float('nan')
missing_score == missing_score
```

结果: False. NaN (缺失/无效的数) 定义为不等于任何东西, 连自己都不等于。用 `math.isnan(x)` 检测。

9.

```
total = 7    # two students' scores added up
total / 2
```

结果: 3.5. / 永远是浮点 (真) 除法, 所以平均值算出来是 3.5。想要整数用 //。

10.

```
penalty = -7    # points to spread over 2 assignments
penalty // 2
```

结果: -4. // 向负无穷取整, 不是向零, 所以 -7 // 2 == -4 (每人扣 4 分, 不是 3 分)。

11.

```
score = '5'    # read from a CSV
score == 5
```

结果: False. Python 不做文本/数字的自动转换, CSV 里的字符串和数字就是不相等 (也不报错)。先转换: `int(score) == 5`。

12.

```
score = '5'  
5 > score
```

结果: `TypeError: '>' not supported between instances of 'int' and 'str'`. 数字和文本排大小会抛 `TypeError`——不存在合理的顺序。先转换: `5 > int(score)`。

13.

```
scores_list = [88, 91]  
scores_tuple = (88, 91)  
scores_list == scores_tuple
```

结果: `False`. 列表永远不等于元组, 内容一样也不行。

14.

```
passed = True  
type(passed) is int
```

结果: `False`. `type()` 是精确匹配, 而 `passed` 的类型是 `bool`, 不是 `int`。要认子类就用 `isinstance`。

15.

```
passed = True  
isinstance(passed, int)
```

结果: `True`. `isinstance` 尊重子类关系, 而 `bool` 确实是一种 `int`。

16.

```
answer = '0' # what a student typed on the survey  
bool(answer)
```

结果: `True`. 任何非空字符串都是真值, 包括 '0' 和 'False'。只有空字符串是假值——所以问卷文本要先转换再判断。

17.

```
submissions = []  
bool(submissions)
```

结果: `False`. 空容器 (`[]`、`{}`、`''`、`0`、`None`) 都是假值, 所以 `if submissions:` 读作“有没有内容?”。

18.


```
id_from_csv = int('257')
id_from_form = int('257') # the same student ID, parsed twice
id_from_csv is id_from_form
```

结果: `False`. CPython 预缓存了小整数（-5..256），在那个范围里 `is` 会碰巧看似 `True`；运行时构造的 257 是两个不同对象。比值永远用 `==`，绝不用 `is`。

第 3 课

控制流：条件与循环

第一部分——条件语句

```
if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
else:
    grade = "F"
```

缩进划定代码块。只有**第一个**为真的分支会执行。

if 与 elif ——不可互换

```
# BUG：三个独立的 if — 95 分会把三个标签全打出来
if score >= 60: print("passing")
if score >= 80: print("strong")
if score >= 90: print("excellent")
```

- 并排的 `if` 是**互相独立的问题**——条件重叠时会全部触发。
- `elif` 把它们变成一个**带分支的问题**：第一个命中即止，其余跳过。
- 只有当多个条件**确实可以同时成立时**，才用并排的 `if`。

比较运算 + 链式比较

`==` `!=` `<` `<=` `>` `>=`

```
0 <= score <= 100      # 链式 — 读起来像数学
```

□ 不必写 `score >= 0 and score <= 100` ——链起来。

逻辑运算符（附一个坑）

```
passed and submitted    # 两者都
late or excused          # 任一
not flagged              # 取反
```

短路求值： `a and b` 在 `a` 为假时直接跳过 `b`。而且 `and / or` 返回的是操作数，不是布尔值：

```
5 and 0          # ? ← 先预测，再到下方陷阱区揭晓
"" or "N/A"      # 默认值惯用法
```

所以写 `if x:` ——永远不要写 `if x == True`。

第二部分——循环

```
for s in scores:          # 直接遍历每个元素
    print(s)

for i in range(5):        # 0,1,2,3,4
    print(i)

while not done:           # 条件翻转前一直重复
    ...
```

△ `range(1, 5)` → `1,2,3,4` —— **不含终点**（差一错误！）。`for _ in range(3):` —— `_` 是“只管重复、不用这个值”的约定写法。

break / continue

```
for x in data:
    if x is None:
        continue          # 跳过这一个
    if x == "STOP":
        break             # 彻底离开循环
    process(x)
```

别再摆弄下标：enumerate 与 zip

```
for i, name in enumerate(names):      # 序号 + 值
    print(i, name)

for name, score in zip(names, scores): # 两个列表并排走
    print(name, score)
```

□ 一旦写出 `range(len(x))`，停下——改用 `enumerate / zip`。

输入校验循环（以后到处都用得上）

```
while True:
    raw = input("Score 0-100: ")
    if raw.isdigit() and 0 <= int(raw) <= 100:
        score = int(raw)
        break
    print("Try again.")
```

轮到你了

examples/session-03/practice.md :

1. 分数分档器——测试边界值（89.999 / 90 / 90.001）。
2. 计算全班平均分，并用 `zip` 给每人标注 PASS/FAIL。
3. 一个健壮的“问到对为止”循环。
4. 双重标签 bug——三个本该合成一架梯子的并排 `if`。

更进一步

更多控制流

三元表达式——一行版的小 `if`

```
label = "pass" if score >= 60 else "fail"
print(f"{name}: {'✓' if attended else '✗'}")
```

只用于微小的选择。要读两遍才懂的，就写成真正的 `if`。

把判断本身返回出去

```
def is_passing(score):          # 笨拙
    if score >= 60:
        return True
    return False

def is_passing(score):          # Pythonic — 比较结果本来就是 bool
    return score >= 60
```

然后直接裸用：`if is_passing(s):`——永远不要写 `if is_passing(s) == True:`。□ 一切“是/否”辅助函数都一样：`is_valid`、`is_missing`、`is_full`——每个一行 `return`。

match/case——更好读的梯子 (3.10+)

```
match answer:
    case 5 | 4:
        label = "agree"
    case 3:
        label = "neutral"
    case 1 | 2:
        label = "disagree"
    case _:
        label = "invalid" # 默认分支
```

当每个分支都在拿同一个值对比字面量时，`match` 比 `elif` 梯子好读。

for/else——不用旗标变量的查找

```
for s in scores:
    if s < 60:
        print("first failing score:", s)
        break
else:
    print("everyone passed") # 只在循环没有 break 时运行
```

这个 `else` 属于 `for`。不需要 `found = False` 那套记账。

嵌套循环

```
for section in ["A", "B"]:
    for student in roster:
        print(section, student)
```

- 外层每走一步，内层完整跑一遍（ $2 \times 4 = 8$ 行输出）。
- `break` 只跳出内层——想两层都跳，就把循环放进函数里 `return`。

给你正在写的模式起个名字

模式	骨架
累加器	<code>total = 0 ... total += x</code>
计数器	<code>n = 0 ... n += 1</code>
迄今最优	<code>best = xs[0] ... if x > best: best = x</code>
哨兵	<code>while True: ... if raw == "done": break</code>

认得这些模式，“对着空编辑器发呆”就变成了“挑一副骨架”。

轮到你了——第二轮

`examples/session-03/practice.md` → **In class — going deeper** : `match/case` 重写、`for/else` 查找、不用 `max()` 的迄今最优，以及 "把判断返回出去" 的重写。

陷阱回顾

- `if x == True` → 直接 `if x:` ; 判断 `None` 用 `x is None` (不是 `== None`)。
- 并排的 `if` 是独立问题——会全部触发；`elif` 才是一架梯子。
- `=` (赋值) vs `==` (比较)——经典手滑。
- `range(1, 5)` 不含 5；测试你的边界值。
- 不要边遍历边修改列表；用 `enumerate/zip` 替代 `range(len(...))`。

(速查表里还有：`match/case`、三元表达式、`for/else`。)

小结

你已经能干净利落地分支与循环。下一课：第 4 课——数据结构。

课后作业（第 4 课之前）

课外完成——不计入课堂时间。完整题目 + 参考答案：`examples/session-03/practice.md` → *Homework*。

1. 出勤标注器 —— 对一系列百分比使用链式比较 + `elif` 梯子。
2. 猜数字游戏 —— 带输入校验和次数统计的 `while` 循环。
3. 闰年判断 —— 一个布尔表达式搞定，用刁钻年份验证。

练习与作业

课堂练习

任务 1 —— 分数分档器

写 `letter_grade(score)`，按 90/80/70/60 的界限返回 A/B/C/D/F，超出 0–100 返回 `"Invalid"`。测试边界值：90、89.999、0、100、-5、101。

任务 2 —— 布尔逻辑

1. `5 and 0` 是什么？`"" or "N/A"` 呢？为什么它们不是 `True/False`？
2. 用 Pythonic 的方式重写 `if attended == True:`。

任务 3 —— 平均分 + 及格/不及格（循环）

给定 `names = ["Ana", "Ben", "Cara", "Dev"]` 和 `scores = [91, 58, 73, 64]`：

1. 用循环和累加变量算平均分。
2. 用 `zip` 打印 `"<name>: PASS" (≥60)` 或 `"<name>: FAIL"`。
3. 用 `sum(s >= 60 for s in scores)` 数及格人数。

任务 4 —— 输入校验循环

写一个真正的 `while True:` 提示循环，直到用户输入 0–100 的整数才停。

任务 5 —— 陷阱检查

这段代码为什么会跳过元素？怎么修？

```
xs = [1, 2, 3, 4]
for x in xs:
    if x % 2 == 0:
        xs.remove(x)
```

任务 6 —— 双重标签 bug

这段代码本想只打印一个标签，95 分却打了三个。解释原因，然后在不改任何条件的前提下修好它：

```
if score >= 60: print("passing")
if score >= 80: print("strong")
if score >= 90: print("excellent")
```

加餐 —— Python 惯用法速练

盖住 `# ->` 后面的答案，逐行预测，再运行。

```
nums = [80, 92, 45]
print(all(n >= 60 for n in nums))    # -> False    (45 不及格)
print(any(n >= 90 for n in nums))    # -> True     (92 达标)
print(92 in nums, 60 in nums)        # -> True False
```

课堂练习——更进一步（第二小时）

E1 —— 改写成链式比较

各用一个链式比较重写：

1. `if x >= 0 and x <= 100:`
2. `if lo < value and value < hi:`
3. `if a == b and b == c:`

E2 —— 倒计时

打印 10 → 1 再打印 Go! ——先用 `range` , 再用 `while` 。两端的差一错误都要当心。

D1 —— `match/case` 重写

用 `match/case` 重写你的李克特分类器：5/4 → "agree"、3 → "neutral"、1/2 → "disagree"、其他 → "invalid"。这里哪个版本更好读——为什么？

D2 —— `for/else` 查找

在 [91, 73, 84, 58, 90] 中找出第一个低于 60 的分数并打印；如果没有，打印 "everyone passed" ——不许用 `found` 旗标变量。

D3 —— 一趟找出最高和最低

在对 [73, 91, 58, 84] 的一次循环里同时跟踪迄今最高和迄今最低（不用 `max()` / `min()` ）。两个都打印出来。

D4 —— 把判断本身返回出去

把下面每个改写成单行 `return` ，然后挑一个在 `if` 里裸用（不写 `== True` ）：

```
def is_passing(score):
    if score >= 60:
        return True
    else:
        return False

def is_full_class(roster):
    if len(roster) >= 30:
        return True
    return False
```

课后作业（第 4 课之前）

约 30–45 分钟，课外完成——不计入课堂时间。先都试一遍，再看答案。

H1 —— 出勤标注器

写 `label(pct)` → "perfect"（恰好 100）、"good"（≥ 80）、"at risk"（≥ 50），其余 "critical"；超出 0–100 返回 "invalid"。对 [100, 92.5, 80, 79.9, 50, 12, -3, 104] 循环打印 `pct -> label`。测试每个边界。

H2 —— 猜数字游戏

设 `secret = 37`。循环：读入一个猜测（校验它是 1–100 的整数——复用今天的校验循环模式），打印 `higher` / `lower` / `got it in N tries`。统计次数。

H3 —— 闰年判断

`is_leap(year)` : 能被 4 整除, 但整百年除外, 除非能被 400 整除——写成一个布尔表达式。验证 :
2024 → True、1900 → False、2000 → True、2026 → False。

参考答案

课堂练习

```
# 1
def letter_grade(score):
    if not 0 <= score <= 100: return "Invalid"
    for cutoff, g in [(90, "A"), (80, "B"), (70, "C"), (60, "D")]:
        if score >= cutoff: return g
    return "F"
# 90->A, 89.999->B, 0->F, 100->A, -5->Invalid, 101->Invalid

# 2
# 5 and 0 -> 0 ; "" or "N/A" -> "N/A" (and/or 返回操作数, 不是布尔值)
result = "pass" if attended else "absent" # 判断就写 `if attended:`

# 3
total = 0
for s in scores: total += s
print(total / len(scores)) # 71.5
for name, score in zip(names, scores):
    print(f"{name}: {'PASS' if score >= 60 else 'FAIL'}")
print("passes:", sum(s >= 60 for s in scores)) # 3

# 4
while True:
    raw = input("Score 0-100: ")
    if raw.isdigit() and 0 <= int(raw) <= 100:
        print("Got", int(raw)); break
    print("Try again.")

# 5 边遍历边删除会让下标错位, 元素被跳过。
xs = [x for x in xs if x % 2 != 0] # 改为构建新列表 -> [1, 3]

# 6 三个独立的 if 是三个互不相干的问题 — 95 对三个都答"是"。
# 串成梯子, 最具体的放最前, 只有第一个命中的运行:
if score >= 90:
    print("excellent")
elif score >= 80:
    print("strong")
elif score >= 60:
    print("passing")
```

课堂练习——更进一步

```
# E1
0 <= x <= 100
lo < value < hi
a == b == c

# E2
for n in range(10, 0, -1):    # 从 10 开始, 到 0 之前停, 步长 -1
    print(n)
print("Go!")

n = 10
while n >= 1:
    print(n)
    n -= 1
print("Go!")

# D1
def likert_label(answer):
    match answer:
        case 5 | 4:
            return "agree"
        case 3:
            return "neutral"
        case 1 | 2:
            return "disagree"
        case _:
            return "invalid"
# 这里 match 胜出: 每个分支都在拿同一个值对比字面量。

# D2
for s in [91, 73, 84, 58, 90]:
    if s < 60:
        print("first failing:", s)
        break
else:
    print("everyone passed")

# D3
scores = [73, 91, 58, 84]
best = worst = scores[0]
for s in scores[1:]:
    if s > best:
        best = s
    if s < worst:
        worst = s
print("best:", best, "worst:", worst)    # 91 58

# D4
def is_passing(score):
    return score >= 60                # 比较结果本来就是 bool

def is_full_class(roster):
    return len(roster) >= 30
```

```
if is_passing(72):                                # 裸用 — 永远别写 `== True`
    print("through!")
```

课后作业

```
# H1
def label(pct):
    if not 0 <= pct <= 100:
        return "invalid"
    if pct == 100:
        return "perfect"
    if pct >= 80:
        return "good"
    if pct >= 50:
        return "at risk"
    return "critical"

for pct in [100, 92.5, 80, 79.9, 50, 12, -3, 104]:
    print(pct, "->", label(pct))
# 100 perfect · 92.5 good · 80 good · 79.9 at risk · 50 at risk · 12 critical ·
# -3 invalid · 104 invalid

# H2
secret, tries = 37, 0
while True:
    raw = input("Guess 1-100: ")
    if not (raw.isdigit() and 1 <= int(raw) <= 100):
        print("Whole number 1-100, try again.")
        continue
    tries += 1
    guess = int(raw)
    if guess < secret:
        print("higher")
    elif guess > secret:
        print("lower")
    else:
        print(f"got it in {tries} tries")
        break

# H3
def is_leap(year):
    return year % 4 == 0 and (year % 100 != 0 or year % 400 == 0)

print(is_leap(2024), is_leap(1900), is_leap(2000), is_leap(2026))
# True False True False
```

陷阱 —— 先预测，再核对

盖住结果，逐个预测，再自己核对。

1.

```
typed_name = '' # the user left the box blank
typed_name or 'Anonymous'
```

结果: 'Anonymous'. `or` 返回第一个真值操作数（否则返回最后一个），`and` 返回第一个假值——不是布尔值。这就是默认值惯用法。

2.

```
weeks = range(1, 5)
list(weeks)
```

结果: [1, 2, 3, 4]. `range(start, stop)` 在 `stop` 之前停下，这里是第 1–4 周。

3.

```
scores = [55, 92, 78]
all(s >= 60 for s in scores)
```

结果: `False`. `all()` 只有在每个元素都通过时才为 `True`；55 不及格，所以是 `False`。

第 4 课

数据结构

四种容器，四种职责

类型	语法	可变？	用途
list	[1, 2, 3]	是	有序、会变化的集合
tuple	(1, 2)	否	固定记录 / 坐标
dict	{"k": v}	是	键 → 值查找
set	{1, 2, 3}	是	去重后的唯一元素

列表与切片

```
xs = [10, 20, 30, 40]
xs[0]      # 10      xs[-1]   # 40 (最后一个)
xs[1:3]    # [20, 30] (不含终点)
xs[:2]     # [10, 20]
xs[::-1]   # 反转
xs.append(50); xs.sort()      # 就地修改
```

△ xs.sort() 返回 **None**——它就地排序。想要新列表用 sorted(xs)。

字典 = 带标签的记录

```
student = {"name": "Ana", "gpa": 3.9}
student["name"]      # "Ana"
student.get("major", "N/A") # 带默认值的安全访问
student["major"] = "Ed"    # 新增/更新
for key, val in student.items(): ...
```

字典列表 = 一个数据集 □

```
roster = [
    {"name": "Ana", "score": 91},
    {"name": "Ben", "score": 58},
]
```

每个字典 = 一行/一名受访者；每个键 = 一个变量/一列。在 pandas 登场（第 8 课）之前，这就是你的整洁数据集。

集合：唯一元素、快速成员判断

```
answers = ["yes", "no", "yes", "maybe", "no"]
set(answers)          # {'yes', 'no', 'maybe'} — 去重
"yes" in set(answers) # True，而且非常快
```

适合“不重复的回答有哪些”和“这个 ID 出现过吗？”

推导式

```
[s["score"] for s in roster]          # 列表
[s for s in roster if s["score"] >= 60] # 带过滤
{s["name"]: s["score"] for s in roster} # 字典
[s["score"] // 10 for s in roster]     # 分数段集合
```

读法：表达式，对每个元素，（可选）如果满足条件。

按键排序

```
sorted(roster, key=lambda s: s["score"])          # 升序
sorted(roster, key=lambda s: s["score"], reverse=True) # 降序
```

`lambda s: s["score"]` = “按 score 字段排序”。

陷阱：别名（标签，不是盒子）

```
a = [1, 2, 3]
b = a          # 同一个列表
a.append(4)
b              # ? 🤔 ← 先预测（见下方陷阱区）

b = a.copy()   # ✔ 独立副本
```

`[[0]*3]*3` 造出的是同一行的 3 个引用——要用 `[[0]*3 for _ in range(3)]`。

轮到你了

`examples/session-04/practice.md`：

1. 构建名册（字典列表）；按分数排序。
2. 一行写出 `{name: score}` 字典推导式。
3. 把学生分进 pass/fail 两组。
4. 复现别名陷阱，然后修好它。

更进一步

容器工具箱

解包

```
name, score = ("Ana", 91)          # 元组解包
head, *rest = [10, 20, 30, 40]     # head=10, rest=[20, 30, 40]
a, b = b, a                        # 交换，再看一眼
for name, score in zip(names, scores): ... # zip 循环的本质就是解包
```

字典的强力方法

```
student.get("major", "N/A")        # 带默认值的安全查找
student.pop("temp", None)           # 删除并返回（不存在则给默认值）
groups.setdefault("pass", []).append(name) # 不存在就先创建，再使用
d1 | d2                             # 合并字典（右边优先，3.9+）
for key, val in student.items(): ...
```

`collections.Counter` —— 一行搞定频数

```
from collections import Counter
counts = Counter(["yes", "no", "yes", "maybe", "yes"])
counts # Counter({'yes': 3, 'no': 1, 'maybe': 1})
counts.most_common(1) # [('yes', 3)]
```

- 你那整段“统计问卷答案”的循环，浓缩成一个表达式。它本质上就是个字典。
-

`collections.defaultdict` —— 分组不费事

```
from collections import defaultdict
by_major = defaultdict(list) # 键不存在？先造一个 []
for s in roster:
    by_major[s["major"]].append(s["name"])
```

和 `setdefault` 干同样的活，规模大了更干净。

集合，完整工具箱

```
fall & spring      # 交集：两学期都在
fall | spring      # 并集：任一学期在
fall - spring      # 差集：只在秋季
fall ^ spring      # 对称差：恰好只在一个学期
{1, 2} <= {1, 2, 3} # 子集？True
list(dict.fromkeys(xs)) # 去重且保持顺序（set 做不到）
```

排序，进阶版

```
sorted(roster, key=lambda s: (-s["score"], s["name"]))
# 分数从高到低，同分按姓名 A→Z
```

- 多键排序：让 `key` 返回一个**元组**——逐元素比较（第 2 课！）。
- Python 的排序是**稳定的**：键相同的元素保持原有顺序。

copy VS deepcopy

```
import copy
flat = a.copy()      # 新的外层列表 — 内部对象仍然共享
full = copy.deepcopy(a) # 一路复制到底
```

法则：有嵌套、还要改内层 → `deepcopy`。其余情况 `.copy()` 就够了。

轮到你了——第二轮

`examples/session-04/practice.md` → **In class — going deeper**：用 `Counter` 和 `defaultdict` 重做、一次多键排序，以及保持顺序的去重。

陷阱回顾

- `=` 是别名；要复制用 `.copy()` / `copy.deepcopy()`。
- `.sort()` 返回 `None`（就地排）；`sorted()` 返回新列表。
- 内容相同，列表 ≠ 元组（第 2 课）。
- `dict.get(key, default)` 避免 `KeyError`。

小结

你已经能存储、查找、去重、排序和重塑数据。 **下一课**：第 5 课——函数、作用域与复用。

课后作业（第 5 课之前）

课外完成——不计入课堂时间。完整题目 + 参考答案：[examples/session-04/practice.md](#) → *Homework*。

1. 成绩册字典操作 —— 增、改、安全查、删。
2. 频数统计器 —— 用普通字典统计问卷答案（不许 import）。
3. 修好网格 —— 复现 `[[0]*3]*3` 共享行 bug，再正确地重建。

练习与作业

课堂练习

从这里开始：

```
roster = [
    {"name": "Ana", "score": 91}, {"name": "Ben", "score": 58},
    {"name": "Cara", "score": 73}, {"name": "Dev", "score": 64},
]
```

任务 1 —— 排名

按分数从高到低打印姓名。

任务 2 —— 映射（字典推导式）

一行构建 `{name: score}`。

任务 3 —— 分组

用循环构建 `{"pass": [...姓名...], "fail": [...姓名...]}`。

任务 4 —— 去重

从 `["A", "B", "A", "C", "B"]` 得到不重复的值和它们的个数。

任务 5 —— 别名

演示 `b = roster` 之后 `roster.append({...})` 也会改变 `b`。再把 `b` 变成独立副本，让它不再跟着变。（提示：嵌套字典 → `copy.deepcopy`。）

加餐 —— Python 惯用法速练

盖住 `# ->` 后面的答案，逐行预测，再运行。

```
head, *tail = [10, 20, 30, 40]
print(head, tail)           # -> 10 [20, 30, 40]    (星号解包)
print({"a": 1} | {"b": 2})  # -> {'a': 1, 'b': 2}    (字典合并, 3.9+)
print({1, 2, 3} & {2, 3, 4}) # -> {2, 3}        (集合交集)
print(list(zip(*[(1, 2), (3, 4)]))) # -> [(1, 3), (2, 4)] (转置)
```

课堂练习——更进一步（第二小时）

E1 —— 切片速练

对 `xs = list(range(10))`，预测：`xs[2:5]` · `xs[-3:]` · `xs[:-3]` · `xs[::2]` · `xs[::-1]` · `xs[5:2:-1]`。

E2 —— 两个学期

`fall = {"Ana", "Ben", "Cara"}`、`spring = {"Ben", "Dev"}` —— 谁两个学期都在？任一学期在？只在秋季？（`&`、`|`、`-`）

D1 —— 一行 `Counter`

用 `collections.Counter` 重做答案统计（`["yes", "no", "yes", "maybe", "yes", "no"]`），并用 `.most_common(1)` 打印最高票答案。

D2 —— 用 `defaultdict` 分组

用 `collections.defaultdict(list)` 把 `roster`（带 `major` 字段的字典列表）分组成 `{major: [names]}`。

D3 —— 多键排序

把 `[("Ana", 91), ("Ben", 73), ("Cara", 91)]` 按分数降序、同分按姓名升序排——一次 `sorted()` 调用、一个元组键。

D4 —— 保序去重

从 `["B", "A", "B", "C", "A"]` 得到 `["B", "A", "C"]`（按首次出现的顺序）。为什么普通的 `set()` 保证不了这一点？

课后作业（第 5 课之前）

约 30–45 分钟，课外完成——不计入课堂时间。先都试一遍，再看答案。

H1 —— 成绩册字典操作

从 `gradebook = {"Ana": 91, "Ben": 58}` 开始。依次：加入 Cara (73)；Ben 重交作业 (58 → 68)；查询 Dev 但不许出 `KeyError` (默认 "no record")；删除 Ben；按分数从高到低打印 `name: score` 行。

H2 —— 频数统计器

用普通字典和 `.get(k, 0)` (不许 `import`) 把 `answers = ["yes", "no", "yes", "maybe", "yes", "no"]` 变成 `{"yes": 3, "no": 2, "maybe": 1}`。再用 `max(counts, key=count.get)` 打印最高票。

H3 —— 修好网格

用 `[[0]*3]*3` 建一个 3×3 全零网格，执行 `grid[0][0] = 9`，打印——亲眼看到 bug。再用推导式重建并证明修好了。

参考答案

课堂练习

```
# 1
print([s["name"] for s in sorted(roster, key=lambda s: s["score"], reverse=True)])
# ['Ana', 'Cara', 'Dev', 'Ben']

# 2
name_to_score = {s["name"]: s["score"] for s in roster}

# 3
groups = {"pass": [], "fail": []}
for s in roster:
    groups["pass" if s["score"] >= 60 else "fail"].append(s["name"])

# 4
vals = ["A", "B", "A", "C", "B"]
distinct = set(vals); print(distinct, len(distinct))    # {'A', 'B', 'C'} 3

# 5
import copy
b = roster                                           # 别名
roster.append({"name": "Eve", "score": 80})
# b 里现在也有 Eve。要保持独立：
b = copy.deepcopy(roster)                          # 之后 roster 的改动不再影响 b
```

课堂练习——更进一步

```
# E1
xs = list(range(10))
xs[2:5]      # [2, 3, 4]          (不含终点)
xs[-3:]      # [7, 8, 9]         (最后三个)
xs[:-3]      # [0, 1, 2, 3, 4, 5, 6]
xs[::-2]      # [0, 2, 4, 6, 8]   (隔一个取一个)
xs[::-1]      # 反转的副本
xs[5:2:-1]    # [5, 4, 3]         (倒着走, 不含终点)

# E2
fall & spring  # {'Ben'}          — 两学期都在
fall | spring  # 四个名字全部     — 任一学期
fall - spring  # {'Ana', 'Cara'}  — 只在秋季

# D1
from collections import Counter
counts = Counter(["yes", "no", "yes", "maybe", "yes", "no"])
print(counts, counts.most_common(1))  # Counter({'yes': 3, ...}) [('yes', 3)]

# D2
from collections import defaultdict
by_major = defaultdict(list)
for s in roster:
    by_major[s["major"]].append(s["name"])

# D3
pairs = [("Ana", 91), ("Ben", 73), ("Cara", 91)]
print(sorted(pairs, key=lambda p: (-p[1], p[0])))
# [('Ana', 91), ('Cara', 91), ('Ben', 73)] — 负号翻转分数, 姓名决胜负

# D4
xs = ["B", "A", "B", "C", "A"]
print(list(dict.fromkeys(xs)))  # ['B', 'A', 'C'] — 字典记得插入顺序;
                                # 集合没有可保的顺序
```

课后作业

```
# H1
gradebook = {"Ana": 91, "Ben": 58}
gradebook["Cara"] = 73          # 新增
gradebook["Ben"] = 68          # 更新
print(gradebook.get("Dev", "no record")) # 安全查询
del gradebook["Ben"]           # 删除
for name, score in sorted(gradebook.items(), key=lambda kv: kv[1], reverse=True):
    print(f"{name}: {score}")    # Ana: 91 / Cara: 73

# H2
answers = ["yes", "no", "yes", "maybe", "yes", "no"]
counts = {}
for a in answers:
    counts[a] = counts.get(a, 0) + 1
print(counts)                  # {'yes': 3, 'no': 2, 'maybe': 1}
print(max(counts, key=counts.get)) # yes

# H3
grid = [[0] * 3] * 3
grid[0][0] = 9
print(grid) # [[9,0,0],[9,0,0],[9,0,0]] — 三个标签贴着同一行（别名！）

grid = [[0] * 3 for _ in range(3)]
grid[0][0] = 9
print(grid) # [[9,0,0],[0,0,0],[0,0,0]] — 各自独立的行
```

陷阱 —— 先预测，再核对

盖住结果，逐个预测，再自己核对。

1.

```
gradebook = [[0] * 3] * 3    # 3 students x 3 assignments
gradebook[0][0] = 9
gradebook
```

结果: `[[9, 0, 0], [9, 0, 0], [9, 0, 0]]`. `[[0]*3]*3` 造出对**同一行**的三个引用，改一处等于改三处。要用 `[[0]*3 for _ in range(3)]`。

2.

```
gpa = {'Ana': 3.9}
gpa.get('Ben')
```

结果: `None`. 键不存在时 `.get()` 返回 `None`（或默认值）而不是抛错——`gpa['Ben']` 则会抛 `KeyError`。

3.

```
ranking = ['Ana', 'Ben', 'Cara']
ranking[::-1]
```

结果: `['Cara', 'Ben', 'Ana']`. 步长为 -1 的切片返回一个反转的**副本**——常见的 Python 惯用法。

第 5 课

函数、作用域与复用

定义与调用

```
def class_average(scores):  
    """Return the mean of a list of scores."""  
    return sum(scores) / len(scores)  
  
class_average([91, 58, 73])    # 74.0
```

□ 函数就是公式/编码方案：同样的输入 → 同样的输出。

return vs print

```
def avg(xs): return sum(xs) / len(xs)    # 把值交回去  
def show(xs): print(sum(xs) / len(xs))  # 只是显示  
  
x = avg([1,2,3])    # x = 2.0  
y = show([1,2,3])   # 打印 2.0 , 但 y 是 None !
```

`print` 是给人看的；`return` 才把值交给下一步。

参数：位置、关键字、默认值

```
def grade(score, scale=100, passing=60):  
    ...  
grade(85)                # 使用默认值  
grade(85, passing=50)     # 关键字参数
```

△ 默认值必须是不可变的（数字、字符串、`None`）——绝不要用 `[]` 或 `{}`。

args / *kwargs

```
def total(*args):          # 任意个位置参数 -> 元组
    return sum(args)
total(1, 2, 3)             # 6

def tag(**kwargs):         # 任意个关键字参数 -> 字典
    return kwargs
tag(name="Ana", gpa=3.9) # {'name': 'Ana', 'gpa': 3.9}

func(*my_list)             # 把列表摊开成参数
func(**my_dict)            # 把字典摊开成关键字参数
```

陷阱：可变默认参数 🤖

```
def add_student(name, roster=[]):    # ✗
    roster.append(name)
    return roster

add_student("Ana")    # ?
add_student("Ben")    # ? ← 列表会一直留着吗？（见下方陷阱区）
```

默认的 `[]` 只在定义时创建一次。下一页给出修法。

修法：默认用 None

```
def add_student(name, roster=None):  # ✓
    if roster is None:
        roster = []
    roster.append(name)
    return roster
```

法则：默认值想用可变对象？改用 `None`，在函数体内创建。

作用域（LEGB）与全局变量

Python 按此顺序查名字：Local → Enclosing → Global → Built-in。

```
count = 0
def bump():
    count = count + 1    # □ UnboundLocalError
```

一旦给 `count` 赋值，它就成了局部变量。避免 `global`；返回新值再重新赋值。

文档字符串与类型标注

```
def class_average(scores: list[float]) -> float:
    """Return the arithmetic mean of `scores`."""
    return sum(scores) / len(scores)
```

类型标注表达意图。运行时并不强制执行（`mypy` 可以检查）。大型项目会规范文档字符串字段（`:param:`、`:return:`、`:raises:`），这样 **Sphinx** 之类的工具能直接从代码生成文档网站。

轮到你了

`examples/session-05/practice.md`：

1. 一个小型成绩函数库（带文档字符串 + 类型标注）。
2. 复现可变默认参数 bug，然后修好它。
3. 用 `*args` 写 `summary(*scores)`。

更进一步

函数也是值

函数是对象

```
f = letter_grade          # 不加 () — 函数本体
f(85)                     # "B"
sorted(roster, key=grade_of)  # 第 4 课起你就在传函数了
stats = {"mean": class_average, "max": max}
stats["mean"]([91, 58, 73])  # 按名字调度
```

`key=lambda ...` 从来不是魔法——你一直在把函数递给 `sorted`。

`lambda`，说实话

```
lambda s: s["score"]      # 只能是一个表达式，没有语句和文档字符串
```

作为一次性的 `key=` 很完美。一旦需要第二行、或需要名字才能看懂——就升级成 `def`。

闭包：会记事的函数

```
def make_curver(bonus):
    def curve(score):
        return min(score + bonus, 100)
    return curve          # 返回函数，`bonus` 随身携带

gentle = make_curver(5)
harsh_year = make_curver(2)
gentle(96)                # 100
```

一个函数工厂。（前面那个“循环里的 lambda”陷阱就是它——捕获的是变量，不是值。）

第一个装饰器

```
def announce(f):
    def wrapper(*args, **kwargs):
        print(f"calling {f.__name__}{args}")
        return f(*args, **kwargs)
    return wrapper

@announce                  # 语法糖：curve = announce(curve)
def curve(score):
    return min(score + 5, 100)
```

装饰器**包裹**一个函数，附加额外行为。你会不停地**用到**它们（`@property`、`@cache`、`@pytest.mark...`）——现在你知道 `@` 是什么了。

锁定调用方式

```
def curve(scores, *, bonus=5):    # * 之后的参数只能用关键字传
    ...
curve(xs, bonus=3)                # ✓ 一目了然
curve(xs, 3)                     # ✗ TypeError — 不许来历不明的位置参数
```

仅关键字参数让一个月后的调用代码依然可读。

能自我检验的文档字符串

```
def class_average(scores):
    """Mean of scores.

    >>> class_average([90, 80, 70])
    80.0
    """
    return sum(scores) / len(scores)
```

`python -m doctest grades.py` 会运行每个 `>>>` 示例并在不符时报错——文档不再会悄悄过期。

轮到你了——第二轮

`examples/session-05/practice.md` → **In class — going deeper**：自己写 `apply_to_all`、一个加分工厂、一个 `@announce` 装饰器，以及一个 `doctest`。

陷阱回顾

- 可变默认参数 → 用 `None`。
- `print` ≠ `return`（忘了 `return` → `None`）。
- 在函数内给全局变量赋值 → `UnboundLocalError`。
- 类型标注不强制执行。

小结

你已经能写出可复用、有文档、可复现的函数。 **下一课**：第 6 课——递归。

课后作业（第 6 课之前）

课外完成——不计入课堂时间。完整题目 + 参考答案：`examples/session-05/practice.md` → *Homework*。

1. 迷你统计库 —— `validate_score`、`curve`、`summarize`，带文档字符串 + 类型标注。
2. `mean_ignoring_none(*values)` —— `*args` 遇上数据清洗。
3. 作用域预测 —— 先说出会打印什么，再运行验证。

练习与作业

课堂练习

任务 1 —— 成绩函数库

写三个带文档字符串和类型标注的函数：

- `class_average(scores: list[float]) -> float`
- `letter_grade(score: float) -> str`（复用第 3 课）
- `pass_rate(scores: list[float], passing: float = 60) -> float`（及格比例，0–1）

`pass_rate` 用布尔求和（回想 `sum(s >= passing for s in scores)`）。

任务 2 —— 复现并修复可变默认值 bug

写 `add_note(text, notes=[])`：追加并返回。连调三次，看列表越长越大。然后用 `None` 模式修好，并证明每次调用都从空列表开始。

任务 3 —— *args 汇总

写 `summary(*scores)`，返回字典 `{"n":..., "mean":..., "max":..., "min":...}`。分别以 `summary(91, 58, 73)` 和 `summary(*my_list)` 两种方式调用。

加餐 —— Python 惯用法速练

盖住 `# ->` 后面的答案，逐行预测，再运行。

```
def f(a, *, b):          # * 之后的参数只能用关键字传
    return a, b
print(f(1, b=2))          # -> (1, 2)
print(f(**{"a": 1, "b": 9})) # -> (1, 9)  (** 把字典摊开成参数)
```

课堂练习——更进一步（第二小时）

E1 —— 仅关键字参数

改写 `letter_grade(score, plus_minus)`，让 `plus_minus` 必须用关键字传（`letter_grade(95, plus_minus=True)`）；按位置传要抛 `TypeError`。

E2 —— 打磨文档字符串

给 `pass_rate` 写一个文档字符串：说明返回什么、`passing` 是什么意思、附一个用法示例。一小段就好，不要废话。

D1 —— 自己写 map

写 `apply_to_all(func, values)`，返回把 `func` 应用到每个值的新列表。用 `abs` 和一个加 5 分的 `lambda` 测试。

D2 —— 函数工厂

写 `make_curver(bonus)`，返回一个“加 `bonus` 分、上限 100”的函数。构造 `gentle = make_curver(5)` 和 `strict = make_curver(1)`，证明两者不同。

D3 —— 第一个装饰器

写 `@announce`：在被包装函数运行前打印 `calling <name> with <args>`。装饰你的 `letter_grade` 并调用。

D4 —— 一个 doctest

给 `class_average` 的文档字符串加两个 `>>>` 示例，然后运行 `python -m doctest your_file.py -v`，看它们通过。

课后作业（第 6 课之前）

约 30–45 分钟，课外完成——不计入课堂时间。先都试一遍，再看答案。

H1 —— 迷你统计库

三个函数，都带文档字符串和类型标注：

- `validate_score(x) -> float` —— 接受 0–100 内的 int/float/数字字符串；否则 `raise ValueError`（并拒绝 `bool` —— 记得第 2 课！），
- `curve(scores: list[float], bonus: float = 5) -> list[float]` —— 加分、封顶 100，不许修改输入列表，
- `summarize(scores: list[float]) -> dict` —— `n / mean / min / max`。

H2 —— `mean_ignoring_none(*values)`

`mean_ignoring_none(90, None, 80, None, 70) → 80.0`。清洗后什么都不剩时返回 `None`，而不是除以零。

H3 —— 作用域预测

这段代码打印什么——哪里会坏？先预测再运行：

```
count = 0

def tally(xs):
    total = 0
    for x in xs:
        total += x
    return total

def bump():
    count = count + 1    # ← 在这里多想想

print(tally([1, 2, 3]))
bump()
```

参考答案

课堂练习

```
def class_average(scores: list[float]) -> float:
    """Mean of scores."""
    return sum(scores) / len(scores)

def letter_grade(score: float) -> str:
    """A/B/C/D/F by 90/80/70/60 cutoffs."""
    for cutoff, letter in [(90, "A"), (80, "B"), (70, "C"), (60, "D")]:
        if score >= cutoff:
            return letter
    return "F"

def pass_rate(scores: list[float], passing: float = 60) -> float:
    """Fraction of scores >= passing (0..1)."""
    return sum(s >= passing for s in scores) / len(scores)

# 任务 2
def add_note(text, notes=None):      # 修好的版本
    if notes is None:
        notes = []
    notes.append(text)
    return notes

# 任务 3
def summary(*scores):
    return {"n": len(scores), "mean": sum(scores)/len(scores),
            "max": max(scores), "min": min(scores)}
print(summary(91, 58, 73))
print(summary(*[91, 58, 73]))
```

课堂练习——更进一步

```
# E1
def letter_grade(score, *, plus_minus=False):    # * 让后面的参数只能用关键字
    ...
# letter_grade(95, True)          -> TypeError
# letter_grade(95, plus_minus=True) -> 正常

# E2
def pass_rate(scores, passing=60):
    """Return the fraction (0..1) of scores at or above `passing`.

    `passing` is the cutoff, 60 by default: pass_rate([70, 50, 90]) -> 0.66...
    """

# D1
def apply_to_all(func, values):
    return [func(v) for v in values]

print(apply_to_all(abs, [-3, 4, -5]))          # [3, 4, 5]
print(apply_to_all(lambda s: min(s + 5, 100), [58, 97])) # [63, 100]

# D2
def make_curver(bonus):
    def curve(score):
        return min(score + bonus, 100)
    return curve

gentle, strict = make_curver(5), make_curver(1)
print(gentle(96), strict(96))    # 100 97

# D3
def announce(f):
    def wrapper(*args, **kwargs):
        print(f"calling {f.__name__} with {args}")
        return f(*args, **kwargs)
    return wrapper

@announce
def letter_grade(score):
    for cutoff, letter in [(90, "A"), (80, "B"), (70, "C"), (60, "D")]:
        if score >= cutoff:
            return letter
    return "F"

print(letter_grade(85))          # calling letter_grade with (85,) -> B

# D4
def class_average(scores):
    """Mean of scores.

    >>> class_average([90, 80, 70])
    80.0
    >>> class_average([100])
    100.0
    """
```

```
    return sum(scores) / len(scores)
# python -m doctest your_file.py -v -> 2 passed.
```

课后作业

```
# H1
def validate_score(x) -> float:
    """Return x as a float score in 0-100, or raise ValueError."""
    if isinstance(x, bool):          # bool 会冒充数字混过 float()!
        raise ValueError("bool is a flag, not a score")
    score = float(x)                 # 坏字符串在这里抛 ValueError
    if not 0 <= score <= 100:
        raise ValueError(f"{score} outside 0-100")
    return score

def curve(scores: list[float], bonus: float = 5) -> list[float]:
    """Return a NEW list with `bonus` added to each score, capped at 100."""
    return [min(s + bonus, 100) for s in scores]

def summarize(scores: list[float]) -> dict:
    """n, mean, min, max of `scores`."""
    return {"n": len(scores), "mean": sum(scores) / len(scores),
            "min": min(scores), "max": max(scores)}

# H2
def mean_ignoring_none(*values):
    clean = [v for v in values if v is not None]
    return sum(clean) / len(clean) if clean else None

print(mean_ignoring_none(90, None, 80, None, 70))    # 80.0
print(mean_ignoring_none(None, None))                # None

# H3
# tally([1, 2, 3]) 打印 6 — `total` 是 tally 的局部变量, 与谁都不冲突。
# bump() 抛 UnboundLocalError: 那次赋值让 `count` 成为 bump 的局部变量,
# 于是右边读到的是一个还不存在的局部名。
# 修法不是 `global` — 返回新值, 在调用处重新赋值。
```


陷阱 —— 先预测，再核对

盖住结果，逐个预测，再自己核对。

1.

```
def enroll(name, roster=[]):  
    roster.append(name)  
    return roster  
enroll('Ana')  
enroll('Ben')
```

结果: `['Ana', 'Ben']`. 默认值在 `def` 时只创建一次，同一个列表跨调用一直存在。用 `roster=None` 然后 `roster = roster or []`。

2.

```
def show_score(s):  
    print(s)  
show_score(91) is None
```

结果: `True`. 打印不是返回。没有 `return` 的函数给出 `None`。

3.

```
makers = [lambda: score for score in [88, 91, 73]]  
[make() for make in makers]
```

结果: `[73, 73, 73]`. 闭包捕获的是变量 `score`，不是它的值；等到调用时循环已把 `score` 留在 73。修法是绑定当前值：`lambda score=score: score`。

第 6 课

递归与递归思维

每个递归的模样

真实例子：一门课的先修链有多深？

```
prereq_of = {"ED700": "ED600", "ED600": "ED500",
             "ED500": "ED400", "ED400": None}    # ED400 没有先修课

def prereqs_deep(course):
    earlier = prereq_of[course]
    if earlier is None:                          # 基例 — 到此停下
        return 0
    return 1 + prereqs_deep(earlier)            # 递归步 — 往回退一门
```

永远是两部分：

- 一个**基例 (base case)** 负责停下，
 - 一个**递归步 (recursive case)** 朝基例靠近。
-

追踪调用栈

`orderings(n)` = 给 n 名学生排名的方式数 = $n!$

```
orderings(3)
= 3 * orderings(2)
= 3 * (2 * orderings(1))
= 3 * (2 * 1)          # 基例返回 1
= 6
```

每次调用都在等待它内部的那次调用。调用层层堆起，再层层返回。

□ 每个未完成的调用是一个**栈帧**——马上就会用到这个概念。

递归 vs 迭代

```
def orderings(n):
    # n 名学生的排名方式数 (n!)
    if n <= 1:
        return 1
    return n * orderings(n - 1)  # ← 必须 RETURN 这次调用

def orderings_loop(n):
    total = 1
    for k in range(2, n + 1):
        total *= k
    return total
```

答案相同。对平铺的计数问题，**循环**通常更清晰。

递归的主场：嵌套数据

```
def deep_sum(obj):
    if isinstance(obj, (int, float)):
        return obj
    if isinstance(obj, dict):
        return sum(deep_sum(v) for v in obj.values())
    if isinstance(obj, (list, tuple)):
        return sum(deep_sum(x) for x in obj)
    return 0

deep_sum([1, [2, [3, 4]], {"a": 6}])  # 16
```

嵌套 JSON、文件夹树、盖楼式回复——单层循环够不到底，递归可以。

陷阱：没有基例

```
def runaway(n):
    return runaway(n + 1)  # 永不停止
```

```
RecursionError: maximum recursion depth exceeded
```

Python **没有尾调用优化**——每次调用都保留栈帧（默认上限 ≈ 1000 ）。太深的递归一定会撞到天花板。

轮到你了

examples/session-06/practice.md :

1. 递归版 `total(scores)` ——先大声说出基例是什么。

2. `flatten([1, [2, [3, 4]], 5])` → 一个平铺列表。
3. `depth(...)` ——列表嵌套有多深？

更进一步

荒野中的递归

记忆化：记住算过的调用

```
def fib(n):
    # 朴素版：fib(35) 会把 fib(5) 重算几千次
    return n if n < 2 else fib(n - 1) + fib(n - 2)

import functools
@functools.cache
def fib(n):
    # 记住每一对（输入 -> 输出）
    return n if n < 2 else fib(n - 1) + fib(n - 2)
```

朴素版 `fib(35)`：数秒。加缓存：瞬间。代码没变——是装饰器（第 5 课！）干的活。当递归会重复解同一个子问题时，缓存它。

分而治之：二分查找

```
def find(sorted_names, target, lo=0, hi=None):
    if hi is None:
        hi = len(sorted_names)
    if lo >= hi:
        return False
    mid = (lo + hi) // 2
    if sorted_names[mid] == target:
        return True
    if sorted_names[mid] < target:
        return find(sorted_names, target, mid + 1, hi)
    return find(sorted_names, target, lo, mid)
```

每次调用把问题**减半**：1 000 个名字 ≈ 10 步，一百万 ≈ 20 步。（前提是数据有序——第 4 课的 `sorted` 派上用场。）

树形数据

```
org = {"name": "Dean",
      "reports": [{"name": "Chair A", "reports": []},
                  {"name": "Chair B",
                   "reports": [{"name": "Prof C", "reports": []}]}]}

def count_people(node):
    return 1 + sum(count_people(r) for r in node["reports"])

count_people(org)      # 4
```

组织架构图、文件夹树、盖楼回复、层级编码——函数的形状照着数据的形状长。诀窍全在这里。

逃出上限：显式栈

```
def flatten_iter(xs):
    out, to_visit = [], list(xs)
    while to_visit:
        x = to_visit.pop(0)
        if isinstance(x, list):
            to_visit = x + to_visit      # 原地拆箱
        else:
            out.append(x)
    return out
```

逻辑相同，没有调用栈帧——嵌套一万层也没问题。数据可能深过约 1000 层时，自备一个待访问列表。

怎么选：循环、递归，还是缓存？

场景	选它
平铺序列	普通循环
自相似 / 嵌套数据	递归
重复的子问题	递归 + @cache
可能非常深	循环 + 显式栈

递归是工具，不是美德——按数据的形状来挑。

轮到你了——第二轮

examples/session-06/practice.md → In class — going deeper：带步数统计的二分查找、组织树计数，以及不用递归的 flatten。

陷阱回顾

- 每个递归都需要**可达的基例**，否则栈会溢出。
- 要 **return** 递归调用——忘了就得到悄无声息的 `None`。
- 递归不免费：每次调用占一个栈帧（没有尾调用优化）。
- 平铺序列和超深任务，用普通**循环**更好。

小结

你已经能解决"以自身定义"的问题——尤其是嵌套数据。 **下一课**：第 7 课——异常与防御式编程。

课后作业（第 7 课之前）

课外完成——不计入课堂时间。完整题目 + 参考答案：`examples/session-06/practice.md` → *Homework*。

1. `sum_digits(n)` —— 对数字而不是列表做递归。
2. `deep_count(obj)` —— 数一数嵌套成绩册里的所有分数。
3. 用自己的话 —— 一段话：什么时候循环是更好的工具？

练习与作业

课堂练习

任务 1 —— 递归求和

用递归（不用循环）写 `total(scores)` 对分数列表求和：`scores[0] + total(rest)`，以空列表为基例。先大声说出基例是什么。测试 `total([91, 58, 73])` 和 `total([])`。

任务 2 —— 递归 vs 迭代

写递归版 `reverse(s)` 反转字符串，再写循环版。你觉得哪个更好读？测试 `reverse("data")`。

任务 3 —— 摊平嵌套数据

写 `flatten(xs)`，把任意深度的列表套列表变成一个平铺列表：`flatten([1, [2, [3, 4]], 5])` → `[1, 2, 3, 4, 5]`。处理嵌套 JSON/导出数据就靠这招。

任务 4 —— 嵌套有多深？

写 `depth(xs)`，返回列表的嵌套深度：`depth([1, [2, [3, [4]]]])` → `4`，`depth([1, 2, 3])` → `1`，`depth(5)` → `0`。

任务 5 —— 陷阱检查

1. 这段为什么抛 `RecursionError` ? 怎么修? `python def f(n): return n + f(n - 1)`
2. 这段返回 `None` 而不是数字——为什么? `python def orderings(n): if n <= 1: return 1 n * orderings(n - 1)`
3. 举一个普通循环比递归更合适的场景。

加餐 —— Python 惯用法速练

一个装饰器就能让指数级的递归瞬间完成——靠记住算过的调用。

```
import functools

@functools.cache          # 记忆化：每个 n 只算一次
def fib(n):
    return n if n < 2 else fib(n - 1) + fib(n - 2)
print(fib(35))            # -> 9227465    (去掉 @cache 再试试.....慢慢等吧)
```

课堂练习——更进一步（第二小时）

E1 —— 递归倒计时

`countdown(3)` 打印 `3 2 1 Go!`。动手之前先说出基例。

E2 —— 纸上追踪

像幻灯片追踪 `orderings(3)` 那样，写出 `total([5, 10, 20])`（任务 1）的完整展开——每一次挂起的调用，然后逐层返回。

D1 —— 数步数的二分查找

写递归版 `find(sorted_names, target)`，顺便统计它看了几个名字。在 7 个名字的有序列表里查找，然后想象 1000 个——大约要几步？为什么？

D2 —— 数组织树

`org = {"name": ..., "reports": [...]}` 任意嵌套。写 `count_people(node)`；再写 `deepest(node)`，返回这棵树有几层。

D3 —— 不用递归摊平

用显式的待访问列表（不写递归调用）重写 `flatten`。在 `[1, [2, [3, 4]], 5]` 上验证与递归版一致——并解释为什么它能扛住一万层的深度。

课后作业（第 7 课之前）

约 30–45 分钟，课外完成——不计入课堂时间。先都试一遍，再看答案。

H1 —— `sum_digits(n)`

`sum_digits(4823)` → 17，用递归。（提示：`n % 10` 是最后一位，`n // 10` 是其余部分。哪个最小的数的数字和一眼便知？）

H2 —— `deep_count(obj)`

数嵌套结构里出现了多少个**数字**（int/float——但 bool 不算！）。处理字典、列表/元组和标量。

`deep_count({"quiz": [90, 85], "final": {"written": 88, "oral": 92}, "note": "great"})` → 4。

H3 —— 用自己的话

一段话：哪种形状的问题更适合普通循环？深度到 1000 左右时递归具体会出什么问题？（说出错误名。）

参考答案

课堂练习

```
# 1
def total(scores):
    if not scores:                # 基例：空列表的和是 0
        return 0
    return scores[0] + total(scores[1:]) # 第一个分数 + 其余的和
print(total([91, 58, 73]), total([]))  # 222 0

# 2
def reverse(s):
    if s == "":                  # 基例：空字符串
        return ""
    return reverse(s[1:]) + s[0]   # 去头反转，再接上头
print(reverse("data"))            # "atad"
# 循环版："".join(reversed(s)) — 平铺字符串通常这样更清楚

# 3
def flatten(xs):
    out = []
    for x in xs:
        if isinstance(x, list):
            out.extend(flatten(x)) # 递归进子列表
        else:
            out.append(x)
    return out
print(flatten([1, [2, [3, 4]], 5])) # [1, 2, 3, 4, 5]

# 4
def depth(xs):
    if not isinstance(xs, list):
        return 0                # 非列表没有嵌套
    return 1 + max((depth(x) for x in xs), default=0)
print(depth([1, [2, [3, [4]]]]), depth([1, 2, 3]), depth(5)) # 4 1 0

# 5
# 1) 没有可达的基例 -> 调用永不停止 -> 栈溢出。
# 修法：在开头加 `if n == 0: return 0` (或 n <= 0)。
# 2) 递归步算了 n*orderings(n-1) 却没有 RETURN，
# 函数走到结尾返回 None。加上 `return`。
# 3) 平铺序列的简单处理循环更好；或者深度可能超过 ~1000 时
# (Python 没有尾调用优化，深递归会 RecursionError，循环则没事)。
```

课堂练习——更进一步

```
# E1
def countdown(n):
    if n == 0:          # 基例
        print("Go!")
        return
    print(n)
    countdown(n - 1)    # 递归步, 朝 0 走

# E2 — 追踪
# total([5, 10, 20])
# = 5 + total([10, 20])
# =    10 + total([20])
# =        20 + total([])
# =            0          <- 基例
# 回程: 20 + 0 = 20 ; 10 + 20 = 30 ; 5 + 30 = 35

# D1
def find(names, t, lo=0, hi=None, steps=1):
    if hi is None:
        hi = len(names)
    if lo >= hi:
        return False, steps
    mid = (lo + hi) // 2
    if names[mid] == t:
        return True, steps
    if names[mid] < t:
        return find(names, t, mid + 1, hi, steps + 1)
    return find(names, t, lo, mid, steps + 1)

roster = sorted(["Ana", "Ben", "Cara", "Dev", "Eve", "Fay", "Gus"])
print(find(roster, "Fay"))    # (True, 2-3 步)
# 1000 个名字 -> 约 10 步: 每步把范围减半 ( $2^{10} = 1024$ )。

# D2
def count_people(node):
    return 1 + sum(count_people(r) for r in node["reports"])

def deepest(node):
    if not node["reports"]:
        return 1
    return 1 + max(deepest(r) for r in node["reports"])

# D3
def flatten_iter(xs):
    out, to_visit = [], list(xs)
    while to_visit:
        x = to_visit.pop(0)
        if isinstance(x, list):
            to_visit = x + to_visit
        else:
            out.append(x)
    return out

print(flatten_iter([1, [2, [3, 4]], 5]))    # [1, 2, 3, 4, 5]
# 没有递归调用 -> 没有栈帧 -> ~1000 帧的上限根本不适用。
```

课后作业

```
# H1
def sum_digits(n):
    if n < 10:                # 一位数：数字和就是它自己
        return n
    return n % 10 + sum_digits(n // 10)

print(sum_digits(4823))      # 17

# H2
def deep_count(obj):
    if isinstance(obj, bool):    # bool < int — 开关标志不算数字（第 2 课！）
        return 0
    if isinstance(obj, (int, float)):
        return 1
    if isinstance(obj, dict):
        return sum(deep_count(v) for v in obj.values())
    if isinstance(obj, (list, tuple)):
        return sum(deep_count(x) for x in obj)
    return 0

print(deep_count({"quiz": [90, 85], "final": {"written": 88, "oral": 92},
                  "note": "great"}))  # 4
```

H3 —— 参考回答：平铺序列（列表求和、逐行处理）用循环最清楚。每个挂起的递归调用都占一个栈帧，而 Python 没有尾调用优化，所以到约 1000 帧就抛 `RecursionError` —— 循环则可以一直跑。递归的价值在于数据本身是嵌套/自相似的时候。

陷阱 —— 先预测，再核对

盖住结果，逐个预测，再自己核对。

1.

```
def grade(n):  
    return grade(n - 1)    # forgot the base case  
grade(3)
```

结果: `RecursionError: maximum recursion depth exceeded`. 每个递归函数都需要基例。没有它，Python 会在递归上限（约 1000 层）处以 `RecursionError` 停下。

2.

```
def curve(score):  
    if score >= 95:  
        return 100  
    score + 5    # looks right...  
curve(80) is None
```

结果: `True`. 第二个分支算出了 `score + 5` 又扔掉了：前面没有 `return`，函数走到结尾就交回 `None`。在递归函数里，同样的手滑会悄悄毒化整条调用链。

3.

```
import sys  
sys.getrecursionlimit()
```

结果: `1000`. 每个未完成的调用都占一个栈帧，CPython 默认给栈设了上限（约 1000），超过就抛 `RecursionError`。递归不免费——平铺的深层工作交给循环。

第 7 课

异常与防御式编程

try / except

```
try:
    n = int(value)
except ValueError:
    n = None          # 处理坏情况
```

代码可能在运行时失败，就包起来。 `except` 捕获指定名字的错误。 `try` 块要保持小——只放可能失败的那一行——这样捕获就不会 误伤你本没打算原谅的代码。

`except ValueError: pass` 是最小化处理——一次看得见的、故意的“跳过”。真要跳过时没问题；但凡你可能会问“跳过了什么？”，就改成记日志。

常见异常类型

异常	何时发生
<code>ValueError</code>	类型对、值不行： <code>int("N/A")</code>
<code>TypeError</code>	类型不对： <code>5 > "5"</code>
<code>KeyError</code>	字典缺键： <code>d["nope"]</code>
<code>IndexError</code>	列表下标越界： <code>xs[99]</code>
<code>ZeroDivisionError</code>	<code>x / 0</code>
<code>FileNotFoundError</code>	<code>open("missing.csv")</code>

try / except / else / finally

```
try:
    n = int(value)
except ValueError:
    print("not a number")
else:
    print("ok:", n)      # 仅当没有异常时
finally:
    print("always runs") # 收尾清理
```

主动抛出

```
def clean_likert(n):
    if not 1 <= n <= 5:
        raise ValueError(f"{n} not in 1-5")
    return n
```

`raise` 是有意抛出异常——如何处理交给调用者决定。

EAFP vs LBYL

```
# LBYL — "三思而后行"
if value.isdigit():
    n = int(value)

# EAFP — "先斩后奏" (Pythonic)
try:
    n = int(value)
except ValueError:
    n = None
```

两者都合法。当"事先检查"困难或有竞态时，EAFP 更亮眼。

assert (开发者检查，不是输入校验)

```
assert len(scores) > 0, "scores must not be empty"
```

给你自己开发时做的合理性检查。它可以被关闭（`python -O`），所以**绝不要用** `assert` 校验不可信输入——用 `raise`。

第一个 pytest 测试

```
# clean.py
def clean_likert(n):
    if not 1 <= n <= 5:
        raise ValueError("1-5 only")
    return n

# test_clean.py
import pytest
from clean import clean_likert

def test_valid():
    assert clean_likert(3) == 3
def test_invalid():
    with pytest.raises(ValueError):
        clean_likert(9)
```

运行： `pytest`

轮到你了

`examples/session-07/practice.md`：

1. `safe_int(value)`：返回 `int` 或 `None`。
2. 清洗一列脏问卷数据：收集有效值 + 一份拒收记录。
3. 写一个 `pytest` 测试。

更进一步

更稳健的程序

异常家族树

```
Exception
├── ValueError          ├── KeyError / IndexError (LookupError)
├── TypeError          └── OSError (FileNotFoundError, ...)
```

```
try:
    n = int(raw)
except (ValueError, TypeError):    # 一次捕获多个具体异常
    ...
```

顺序很重要：`except` 子句自上而下匹配——先具体、后宽泛；末尾的 `except Exception as e`：只用来记录并停止，绝不用来无视。

携带证据的自定义异常

```
class SurveyError(ValueError):
    def __init__(self, value, message):
        super().__init__(message)
        self.value = value          # 把证据留下

raise SurveyError(raw, f"{raw!r} is not a 1-5 rating")
```

调用者可以专门捕 `SurveyError`——也可以宽泛地捕 `ValueError`——并且仍能看到哪个值出的问题。

raise ... from——留住起因

```
try:
    n = int(cell)
except ValueError as e:
    raise SurveyError(cell, "bad rating cell") from e
```

回溯信息会**两层都显示**：上面是你的领域级错误，下面是真正的底层起因。凌晨调试的未来的你会感谢现在的你。

finally 与清理思维

```
try:
    f = open("data.csv")
    ...
finally:
    f.close()          # 无论上面发生什么都会运行
```

`with open(...)`（第 8 课）正是这套模式的成品包装。法则：谁申请资源，谁保证释放。

诊断信息，logging 胜过 print

```
import logging
logging.basicConfig(level=logging.INFO, format="%(levelname)s: %(message)s")
logging.info("processing %s rows", len(rows))
logging.warning("rejected %r: out of range", raw)
```

- `print` 输出程序的结果；`logging` 记录程序的日记。
- 级别（`DEBUG/INFO/WARNING/ERROR`）让你不删代码就能调低音量。

pytest，进阶版

```
import pytest
from clean import clean_likert

@pytest.mark.parametrize("bad", ["3", True, None, 0, 9])
def test_rejects(bad):
    with pytest.raises(ValueError):
        clean_likert(bad)
```

一个测试、五个输入、五条独立报告。套路：**准备、执行、断言**——并且测边界（临界值、错误类型），而不是舒适区。

只会 `print` 的函数没法断言。让函数返回值、在边缘处打印——这才是可测试的代码（第 5 课 `return` vs `print` 的回报）。

轮到你了——第二轮

`examples/session-07/practice.md` → **In class — going deeper**：修一个 `except` 顺序 bug、写带 `raise ... from` 的 `SurveyError`、把拒收记录换成 `logging`，以及参数化你的测试。

陷阱回顾

- 绝不裸写 `except`：——写明异常名。
- 别捕得太宽，别悄悄吞掉错误。
- `assert` ≠ 输入校验（用 `raise`）。
- 捕你预料中的那个具体错误。

小结

你已经能校验脏输入，并在该失败时大声失败。 **下一课**：第 8 课——文件与研究数据。

课后作业（第 8 课之前）

课外完成——不计入课堂时间。完整题目 + 参考答案：`examples/session-07/practice.md` → *Homework*。

1. `ask_int(prompt, lo, hi)` —— 用 EAFP 重写第 3 课的输入校验循环。
2. 再加三个 `pytest` 用例 —— `clean_likert` 的边界情况（`True`、`"3"`、`None`）。
3. 报错分诊 —— 五条真实报错信息：说出异常类名和修法。

练习与作业

课堂练习

任务 1 —— `safe_int`

写 `safe_int(value)`：返回 `int(value)`，失败时返回 `None`。用 `"42"`、`"N/A"`、`""`、`None`、`3.0` 测试。

任务 2 —— 清洗一系列问卷数据

给定 `raw = ["5", "3", "N/A", "7", "", "1", "two", "4"]`，产出：

- `clean` —— 有效的李克特整数（1–5）列表，以及

- `rejected` —— (值, 原因) 对的列表。用一个对越界或非整数抛出 `ValueError` 的 `clean_likert(n)` 来实现。

任务 3 —— 写一个测试

把 `clean_likert` 放进 `clean.py` , 用 `pytest` 写 `test_clean.py` : 一个通过用例 + 一个 `pytest.raises(ValueError)` 用例。运行 `pytest` 。

任务 4 —— 讨论

裸 `except` : 为什么危险? 举一个它会藏起来、而你更想看到的错误。

加餐 —— Python 惯用法速练

盖住 `# ->` 后面的答案, 逐行预测, 再运行。

```
class LikertError(ValueError):          # 你自己的异常类型
    pass
print(issubclass(LikertError, ValueError))  # -> True (所以 except ValueError 也接得住)

try:
    assert 1 == 2, "values differ"      # assert: 廉价的内部自检
except AssertionError as e:
    print(e)                            # -> values differ
```

课堂练习——更进一步 (第二小时)

E1 —— 哪个异常?

不运行, 说出每个会抛什么异常: `int("3.5")` · `{"a": 1}["b"]` · `[1, 2][5]` · `1/0` · `open("nope.csv")` · `len(42)`

E2 —— 你自己的异常

定义 `class SurveyError(ValueError)` , 对越界的李克特值抛出它。证明 `except ValueError:` 仍然接得住——这就是子类化在起作用。

D1 —— 修好 `except` 顺序

这段对坏单元格永远打印 `"unexpected"` ——为什么? 怎么修?

```
try:
    n = int(cell)
except Exception:
    print("unexpected")
except ValueError:
    n = None
```

D2 —— 携带证据的 `SurveyError`

写 `class SurveyError(ValueError)`，把出错的值存进 `.value`。在清洗循环里接住 `int()` 的底层 `ValueError`，用 `raise SurveyError(cell, ...) from e` 重新抛出。展示一次两层的回溯。

D3 —— 记日志，别打印

把你的拒收记录换成 `logging`：每个被拒的单元格 `logging.warning`，最终统计 `logging.info`。把格式配置成显示级别名。

D4 —— 参数化

把三个作业测试改写成一个 `@pytest.mark.parametrize` 测试，覆盖 `["3", True, None, 0, 9]`。运行 `pytest -q`。

课后作业（第 8 课之前）

约 30–45 分钟，课外完成——不计入课堂时间。先都试一遍，再看答案。

H1 —— `ask_int(prompt, lo, hi)`

用 EAFP 重写第 3 课的输入校验循环：用 `try: n = int(raw) / except ValueError` 替代 `.isdigit()`——这样 `"-5"` 和 `" 42 "` 也能处理。循环到输入有效为止；返回整数。

H2 —— 再加三个 `pytest` 用例

给 `clean_likert` 补测试：`clean_likert(True)` 要抛错（布尔不是评分！）、`clean_likert("3")` 要抛错（字符串，哪怕是数字样子）、`clean_likert(None)` 要抛错。运行 `pytest -q` 直到全绿。

H3 —— 报错分诊

对每条信息，说出异常类名和一行修法：

1. `invalid literal for int() with base 10: 'N/A'`
2. `unsupported operand type(s) for +: 'int' and 'str'`
3. `'score'`（来自字典查询）
4. `division by zero`
5. `[Errno 2] No such file or directory: 'survy.csv'`

参考答案

课堂练习

```
def safe_int(value):
    try:
        return int(value)
    except (ValueError, TypeError):
        return None

def clean_likert(n):
    if isinstance(n, bool) or not isinstance(n, int):
        raise ValueError(f"{n!r} not an int")
    if not 1 <= n <= 5:
        raise ValueError(f"{n} outside 1-5")
    return n

raw = ["5", "3", "N/A", "7", "", "1", "two", "4"]
clean, rejected = [], []
for r in raw:
    try:
        clean.append(clean_likert(safe_int(r)))
    except ValueError as e:
        rejected.append((r, str(e)))
print(clean)      # [5, 3, 1, 4]
print(rejected)   # [('N/A', ...), ('7', ...), ('', ...), ('two', ...)]
```

```
# test_clean.py
import pytest
from clean import clean_likert
def test_ok():    assert clean_likert(3) == 3
def test_bad():
    with pytest.raises(ValueError):
        clean_likert(9)
```

任务 4：裸 `except:` 连 `KeyboardInterrupt`（Ctrl+C）和你自己笔误产生的 `NameError` 都会接住——拼错的变量名会被悄悄吞掉，你永远看不到那个 bug。永远只接你预料中的那个具体异常。

课堂练习——更进一步

```
# E1
int("3.5")          # ValueError          (int() 只解析整数文本)
{"a": 1}["b"]       # KeyError
[1, 2][5]           # IndexError
1 / 0               # ZeroDivisionError
open("nope.csv")    # FileNotFoundError
len(42)             # TypeError          (整数没有长度)

# E2
class SurveyError(ValueError):
    pass

def clean_likert(n):
    if not 1 <= n <= 5:
        raise SurveyError(f"{n} outside 1-5")
    return n

try:
    clean_likert(9)
except ValueError as e:      # 父类接得住子类
    print("caught:", e)

# D1 — except 子句自上而下匹配; Exception 是 ValueError 的父类,
# 放在前面就先命中, 具体的处理器永远轮不到。先具体、后宽泛:
try:
    n = int(cell)
except ValueError:
    n = None
except Exception as e:
    print("unexpected:", e)
    raise

# D2
class SurveyError(ValueError):
    def __init__(self, value, message):
        super().__init__(message)
        self.value = value

def clean_cell(cell):
    try:
        n = int(cell)
    except ValueError as e:
        raise SurveyError(cell, f"{cell!r} is not a rating") from e
    return n

# 回溯里下层是起因 ValueError, 上层是 SurveyError。

# D3
import logging
logging.basicConfig(level=logging.INFO, format="%(levelname)s: %(message)s")
kept, rejected = [], []
for cell in ["5", "N/A", "3", ""]:
    try:
        kept.append(clean_cell(cell))
    except SurveyError as e:
        rejected.append(e.value)
```

```

        logging.warning("rejected %r", e.value)
    logging.info("kept %d, rejected %d", len(kept), len(rejected))

# D4 — test_clean.py
import pytest
from clean import clean_likert

@pytest.mark.parametrize("bad", ["3", True, None, 0, 9])
def test_rejects(bad):
    with pytest.raises(ValueError):
        clean_likert(bad)

```

课后作业

```

# H1
def ask_int(prompt, lo, hi):
    while True:
        raw = input(prompt)
        try:
            n = int(raw)          # EAFP: 直接试转换
        except ValueError:
            print("A whole number, please.")
            continue
        if lo <= n <= hi:
            return n
        print(f"Between {lo} and {hi}, please.")

# H2 — test_clean.py
import pytest
from clean import clean_likert

def test_bool_rejected():
    with pytest.raises(ValueError):
        clean_likert(True)

def test_str_rejected():
    with pytest.raises(ValueError):
        clean_likert("3")

def test_none_rejected():
    with pytest.raises(ValueError):
        clean_likert(None)

```

H3 —— 分诊：

1. `ValueError` —— 先清洗/转换，或包进 `try/except ValueError`。
2. `TypeError` —— 转换其中一边：`str(n)` 或 `int(s)`。
3. `KeyError` —— 用 `row.get("score")`，或检查表头拼写。
4. `ZeroDivisionError` —— 除之前先挡住空数据的情况。
5. `FileNotFoundError` —— 文件名拼错了（`survey.csv`）；用 `Path(...).exists()` 检查。

陷阱 —— 先预测，再核对

盖住结果，逐个预测，再自己核对。

1.

```
score = '3.0'    # a value from a CSV
int(score)
```

结果: `ValueError: invalid literal for int() with base 10: '3.0'`. `int()` 只解析整数文本；'3.0' 不是合法的整数字面量。用 `int(float(score))`。

2.

```
score = 2.5
round(score)
```

结果: 2. Python 用银行家舍入（一半时取偶数）：`round(2.5) == 2` 而 `round(3.5) == 4`。汇总数据时当心。

3.

```
weight = 0.1 + 0.2 + 0.3    # three grade components
weight == 0.6
```

结果: `False`. 累加各项时浮点误差会累积：0.1+0.2+0.3 是 0.6000000000000001。比较总和用 `math.isclose`，展示时再取整。

4.

```
enrolment = '1_000'
int(enrolment)
```

结果: 1000. Python 接受下划线作数字分隔符，连 `int()` 的文本里也接受。

第 8 课

文件、库与研究数据

用 `with` 打开文件

```
with open("notes.txt") as f:
    text = f.read()
# 到这里文件自动关闭，即使代码崩了也一样
```

`with` = 上下文管理器：替你搭好、拆好资源。永远优先用它，而不是裸的 `open()` / `close()`。

文件模式（当心陷阱）

模式	含义
"r"	读取（默认）
"w"	写入 —— 先把文件清空！
"a"	追加
"r+"	读 + 写

⚠ 用 `"w"` 打开了不该动的文件 → 内容瞬间蒸发。

```
with open("log.txt", "a") as f:
    f.write("cleaned survey.csv\n")
```

追加：写到末尾，绝不清空
写入时 \n 由你自己负责

（`"rb"` / `"wb"` 读写**二进制**——图片、PDF。默认是文本模式。）

读取文本

```
with open("notes.txt") as f:
    whole = f.read()
    # 一整个大字符串
    # 或者
    for line in f:
        print(line.rstrip())
    # 逐行读（省内存）
```

⚠ 文件对象读完一遍就“耗尽”了——想再读要重新打开。

读 CSV：每行一个字典 □

```
import csv
with open("students.csv", newline="") as f:
    for row in csv.DictReader(f):
        print(row["name"], row["score"])    # row 是以表头为键的字典
```

`csv.DictReader` 把每一行变成字典——正是第 4 课的“字典列表”数据集。（`newline=""` 防止 Windows 上出现空行。）

你可以自己用 `.split(",")` 拆行——直到某个字段里也有逗号。这个问题，加上引号和表头，就是 `csv` 存在的理由。

写 CSV

```
with open("summary.csv", "w", newline="") as f:
    w = csv.DictWriter(f, fieldnames=["name", "score"])
    w.writeheader()
    w.writerow({"name": "Ana", "score": 91})
```

研究者常用的库

```
import statistics
statistics.mean(xs); statistics.median(xs); statistics.stdev(xs)

import random
random.choice(xs); random.randint(1, 6); random.shuffle(xs)

from datetime import date
date.today()

from pathlib import Path
Path("students.csv").exists()
```

```
# pip install cowsay          # PyPI 上有约 50 万个包
import cowsay
cowsay.cow("pip install anything")
```

`pip install <名字>`，然后 `import`——第三方库的全部故事就这么多。（对，连搞笑的包也有：重点是安装易如反掌。）

轮到你了

`examples/session-08/practice.md`（用到 `survey.csv`）：

1. 读 `students.csv` ; 用 `statistics.mean` 打印全班平均分。
2. 计算问卷各题的均值 ; 写出 `survey_summary.csv` 。

更进一步

真实的数据工作

`pathlib` , 完整巡礼

```
from pathlib import Path
data = Path("data")
f = data / "students.csv"          # / 拼接路径, 任何系统都行
f.exists(), f.stat().st_size       # 存在吗? 多大?
list(data.glob("*.csv"))           # 这里的每个 CSV
list(data.rglob("*.csv"))          # .....连同所有子文件夹里的
data.mkdir(exist_ok=True)
notes = f.with_suffix(".txt").read_text() # 小文件: 一行搞定
```

编码 (Excel 之坑)

```
open("survey.csv", newline="", encoding="utf-8")          # 明说出来
open("from_excel.csv", newline="", encoding="utf-8-sig") # 吃掉 Excel 的 BOM
```

- 乱码 (é 变 Ã©) = 字节被用错误的编码读了。
- Excel 常存成 `cp1252` 或带 BOM 的 UTF-8——后者用 `utf-8-sig` 处理。

`csv` , 进阶版

```
csv.reader(f)          # 每行是列表 (没有表头/表头重复时用)
csv.DictReader(f)       # 每行是字典 (默认首选)
w.writerows(report)     # 一次写入多个字典
csv.DictReader(f, delimiter=";") # 欧洲版 Excel 导出
```

嵌套数据用 JSON

```
import json
snapshot = {"course": "ED101",
            "items": {"q1": {"mean": 4.2, "n": 28}, "q2": {"mean": 3.8, "n": 27}}}
Path("snapshot.json").write_text(json.dumps(snapshot, indent=2))
back = json.loads(Path("snapshot.json").read_text())
```

CSV 是矩形的；JSON 能嵌套。True/None 会写成 true/null——JSON 是另一门语言，json.load 负责翻译回来。

API：同样的 JSON，来自网络（尝一口）

```
# pip install requests
import requests

r = requests.get("https://itunes.apple.com/search",
                 params={"term": "python", "media": "ebook", "limit": 3})
data = r.json()  # 响应体，解析成字典/列表
for item in data["results"]:
    print(item["trackName"])
```

所谓“调用 API”不过是：请求一个 URL，解析它的 JSON——同一套 json 本领，换了个来源。（需要联网；本课程立足离线文件。）

datetime——日期也是数据

```
from datetime import date, datetime
d = datetime.strptime("2026-07-06", "%Y-%m-%d")  # 解析文本 -> datetime
d.strftime("%b %d")  # 格式化 -> "Jul 06"
(date(2026, 7, 6) - date(2026, 1, 5)).days  # 182 — 日期能做算术
date.fromisoformat("2026-07-06")  # ISO 快捷方式
```

strptime = 解析（字符串 → 日期），strftime = 格式化（日期 → 字符串）。

random，可复现版

```
import random
random.seed(42)  # 每次运行同样的“随机”——方法部分狂喜
random.choice(roster)  # 抽一个
random.sample(roster, 3)  # 抽三个，不重复
random.shuffle(order)  # 就地打乱——随机分组
```

设了种子，随机分析就可复现——重跑脚本会抽出同一批样本。

带参数的脚本：`sys.argv`

```
# report.py — 运行方式：python3 report.py survey.csv
import sys

if len(sys.argv) != 2:
    # argv[0] 是脚本自己的名字
    sys.exit("Usage: python3 report.py <file.csv>")
filename = sys.argv[1]
```

- `sys.argv` 是命令行给的**字符串列表**——可以 `len()`、可以切片（`sys.argv[1:]` = 真正的参数）。
- 先设防、再信任：缺参数就是一个待爆的 `IndexError`；`sys.exit("提示")` 会停止程序并打印用法。
- 选项多起来（`--top 5 --verbose`）就升级到标准库的 `argparse`。

二进制文件实战：用 Pillow 处理图片

```
# pip install pillow
from PIL import Image

before = Image.open("chart_2025.png")    # 二进制文件，解码成像素
after = Image.open("chart_2026.png")
before.save("growth.gif", save_all=True,
            append_images=[after],        # 多帧 -> 动图 GIF
            duration=500, loop=0)         # 每帧毫秒数；0 = 无限循环
```

- 文本模式把字节解码成字符；**二进制格式**（图片、PDF）需要懂行的库——图片这行的专家是 Pillow。
- 三个调用 = 一张结果图表的“前后对比”动图，直接进幻灯片。

pandas 预告，加长版

```
import pandas as pd
df = pd.read_csv("students.csv")
df["score"].describe()                # 计数/均值/标准差/四分位
df.groupby("major")["score"].agg(["mean", "count"])
df[df["score"] < 60]                  # 筛选行
df.to_csv("report.csv", index=False)
```

五行 = 今天一整课。那是下一门课——而现在你知道每一行底下发生了什么。

轮到你了——第二轮

`examples/session-08/practice.md` → **In class — going deeper** : `pathlib` 盘点、报名日期的日期运算、带种子的抽样，以及 CSV → JSON。

陷阱回顾

- `"w"` 会不声不响地覆盖——确认清楚文件名。
- 用 `csv` 模块 → 打开时带 `newline=""`。
- 文件读一遍就耗尽；重读要重新打开。
- 非 ASCII 文本记得指定 `encoding="utf-8"`。

小结

你已经能加载、汇总并写出真实的研究数据。 **下一课**：第 9 课——正则表达式与文本清洗。

课后作业（第 9 课之前）

课外完成——不计入课堂时间。完整题目 + 参考答案：`examples/session-08/practice.md` → *Homework*。

1. **出勤报告** —— 读入、清洗、汇总、写出 CSV——完整管道。
2. **JSON 往返** —— 用 `json.dump` 保存汇总，读回来，验证相等。
3. **你自己的数据** —— 把这条管道对准你研究中的任意一个 CSV。

练习与作业

课堂练习

本文件夹提供：`students.csv`、`survey.csv`。

任务 1 —— 读取并汇总学生

用 `csv.DictReader` 读 `students.csv`。记住值是**字符串**——把 `score` 转成 `int`。用 `statistics` 模块打印全班均值和中位数。

任务 2 —— 按专业求均值

构建 `{major: mean_score}`。（提示：`dict.setdefault(key, []).append(...)`。）

任务 3 —— 清洗并汇总问卷

`survey.csv` 的数字列里混着 `"N/A"` 和空白。对每个 `q*` 题目，只对**有效值**求均值，并统计有效个数。写出 `survey_summary.csv`，列为 `item,mean,n_valid`。

任务 4 —— pandas 预告（可选）

装了 pandas 的话：`pd.read_csv("students.csv")["score"].describe()`。把均值和你手算的对比一下。

陷阱检查

如果不小心用 "w" 模式打开 `students.csv` 再去读，会发生什么？

加餐 —— Python 惯用法速练

盖住 `# ->` 后面的答案，逐行预测，再运行。

```
import json
s = json.dumps({"n": 3, "ok": True})
print(s)                                # -> {"n": 3, "ok": true}    (Python 的 True -> JSON 的 true)
print(json.loads(s)["ok"])              # -> True                (再翻译回 Python 布尔值)
```

课堂练习——更进一步（第二小时）

E1 —— 再加一列

扩展问卷汇总：加一列 `pct_valid`（有可用值的行占比），保留一位小数。

E2 —— pathlib 小巡游

用 `from pathlib import Path`：`students.csv` 存在吗？多少字节（`.stat().st_size`）？列出本文件夹的每个 `.csv`（`.glob`）。

D1 —— pathlib 盘点

用 `Path(".").glob` 和 `.stat().st_size` 把本文件夹每个 `.csv` 及其字节数打印成对齐的行。

D2 —— 日期运算

用 `strptime` 解析 `"2026-01-05"` 和 `"2026-07-06"`，打印两者相差的天数，再用 `strftime` 把第一个日期打印成 `Jan 05, 2026`。

D3 —— 可复现的抽样

先 `random.seed(42)`，再从 `students.csv` 的学生里 `random.sample` 抽 3 人。跑两遍——同样 3 个人吗？论文的方法部分为什么在乎这一点？

D4 —— CSV → JSON

把 `students.csv` 读成分数为 `int` 的字典列表，用 `indent=2` 写成 `students.json`。打开文件看看：引号和数字发生了什么？

D5 —— 从命令行取文件名

把任务 1 改造成 `report.py`：读命令行指定的 CSV（`python3 report.py students.csv`），打印全班均值；参数缺失或多余时输出用法提示并退出。（`sys.argv`、`sys.exit`。）

D6 —— 会动的前后对比

用 Pillow（`pip install pillow`）以 `Image.new("RGB", (60, 60), color)` 生成两帧纯色图，存成一个循环播放的 GIF（`save_all=True`、`append_images=[...]`、`duration=400`、`loop=0`）。打开文件——它在闪。这个二进制文件有多大（`pathlib`）？

课后作业（第 9 课之前）

约 30–45 分钟，课外完成——不计入课堂时间。先都试一遍，再看答案。

H1 —— 出勤报告（完整管道）

自己创建 `attendance.csv`：一列 `name` 加五列 0/1 的 `s1..s5`，约六行——再偷偷放一个脏单元格（`"?"`）。然后：用 `DictReader` 读入，计算每人出勤率（跳过脏值；记住 0/1 整数直接可以求和），用 `DictWriter` 写出 `attendance_report.csv`，列为 `name,rate,n_valid`。全部放在 `with open(...)` 里。

H2 —— JSON 往返

用 `json.dump` 把各题均值存进 `summary.json`（`indent=2`），用 `json.load` 读回，验证 `loaded == original`。再用编辑器打开文件——`True` 变成了什么？

H3 —— 你自己的数据

把这条管道对准你工作里的任何一个 CSV（需要的话从 Excel/表格导出一个）：读入、数行数、算一个均值、打印两行报告。记下你遇到的第一个脏值和你的处理方式——写进你的 bug 日志。

参考答案

课堂练习

见本文件夹的 `demo.py` ——任务 1–3 的完整实现。关键行：

```
scores = [int(s["score"]) for s in students]      # 字符串要转换！
statistics.mean(scores)                          # 75.5

by_major = {}
for s in students:
    by_major.setdefault(s["major"], []).append(int(s["score"]))

def to_int(x):
    try: return int(x)
    except (ValueError, TypeError): return None   # 处理 N/A 和 ""
```

陷阱：用 `"w"` 打开会**立刻把文件清空**——你还没读，数据就没了。读取用 `"r"`（默认值）。

课堂练习——更进一步

```
# E1 — 在问卷题目循环里
vals = [to_int(r[item]) for r in rows]
good = [v for v in vals if v is not None]
writer.writerow({"item": item,
                  "mean": round(statistics.mean(good), 2),
                  "n_valid": len(good),
                  "pct_valid": f"{100 * len(good) / len(vals):.1f}%"})

# E2
from pathlib import Path
p = Path("students.csv")
print(p.exists())                # True (在本文件夹)
print(p.stat().st_size)          # 字节数
print(sorted(Path(".").glob("*.csv"))) # 这里的每个 CSV

# D1
for p in sorted(Path(".").glob("*.csv")):
    print(f"{p.name:<24}{p.stat().st_size:>8,} bytes")

# D2
from datetime import datetime
a = datetime.strptime("2026-01-05", "%Y-%m-%d")
b = datetime.strptime("2026-07-06", "%Y-%m-%d")
print((b - a).days)              # 182
print(a.strftime("%b %d, %Y"))    # Jan 05, 2026

# D3
import csv, random
with open("students.csv", newline="", encoding="utf-8") as f:
    names = [r["name"] for r in csv.DictReader(f)]
random.seed(42)
print(random.sample(names, 3))    # 每次运行同样 3 人 — 种子固定了随机数,
                                   # 你的"随机"抽样因此可复现。

# D4
import json
with open("students.csv", newline="", encoding="utf-8") as f:
    rows = [{**r, "score": int(r["score"])} for r in csv.DictReader(f)]
Path("students.json").write_text(json.dumps(rows, indent=2))
# 文件里：键/字符串带双引号，分数是光秃秃的（真正的 JSON 数字）。

# D5 — report.py
import csv, statistics, sys

if len(sys.argv) != 2:            # argv[0] 是 report.py 自己
    sys.exit("Usage: python3 report.py <file.csv>")

with open(sys.argv[1], newline="", encoding="utf-8") as f:
    scores = [int(r["score"]) for r in csv.DictReader(f)]
print(f"n={len(scores)} mean={statistics.mean(scores):.1f}")

# D6
from PIL import Image
from pathlib import Path
```

```
a = Image.new("RGB", (60, 60), "steelblue")
b = Image.new("RGB", (60, 60), "goldenrod")
a.save("pulse.gif", save_all=True, append_images=[b], duration=400, loop=0)
print(Path("pulse.gif").stat().st_size, "bytes") # 几百字节 — 小,但是真二进制
```

课后作业

```
# H1
import csv

with open("attendance.csv", newline="") as f:
    rows = list(csv.DictReader(f))

report = []
for r in rows:
    marks = []
    for key in list(r)[1:]: # "name" 之后的每一列
        try:
            marks.append(int(r[key]))
        except ValueError:
            pass # 跳过脏单元格——但按无效计
    rate = sum(marks) / len(marks) if marks else 0.0
    report.append({"name": r["name"], "rate": round(rate, 2), "n_valid": len(marks)})

with open("attendance_report.csv", "w", newline="") as f:
    w = csv.DictWriter(f, fieldnames=["name", "rate", "n_valid"])
    w.writeheader()
    w.writerows(report)

# H2
import json

original = {"q1": 4.2, "q2": 3.8, "all_valid": False}
with open("summary.json", "w") as f:
    json.dump(original, f, indent=2)
with open("summary.json") as f:
    loaded = json.load(f)
print(loaded == original) # True — 而文件里 Python 的 False 写成了 JSON 的
                          # false (小写): JSON 是另一门语言。
```

H3 —— 是套路, 不是标准答案: `with` 里 `rows = list(csv.DictReader(f))`, `len(rows)` 数行数, 选一个数字列过 `to_int / to_float` 清洗后求均值。第一个脏值和它抛的异常写进你的 bug 日志。

陷阱 —— 先预测，再核对

盖住结果，逐个预测，再自己核对。

1.

```
import csv, io
row = next(csv.DictReader(io.StringIO('name,score\nAna,91')))
row['score'] + 1
```

结果: `TypeError: can only concatenate str (not "int") to str`. 每个 CSV 值到手都是字符串 —— `'91' + 1` 不是加法，是崩溃。先转换: `int(row['score']) + 1`。

2.

```
import json
json.dumps({'passed': True})
```

结果: `'{"passed": true}'`. JSON 是另一门语言: Python 的 `True` 写成小写的 `true` (`None` 变成 `null`)。 `json.load` 会翻译回来。

3.

```
import io
f = io.StringIO('Ana\nBen\n') # stands in for an open file
first_pass = f.readlines()
f.readlines()
```

结果: `[]`. 文件对象是一个游标，不是容器: 读完一遍后停在末尾，再读返回空。重新打开 (或 `seek(0)`)，或者一次性读进列表。

第 9 课

正则表达式与文本清洗

研究者为什么要在乎

- 在 ID、邮箱、日期污染数据之前先做校验。
 - 从自由文本里提取结构化的片段（编码、姓名、数字）。
 - 清洗、规范开放式问卷回答。
 - **定性编码**的第一遍粗筛（找出匹配某模式的所有回答）。
- 像在语料库里做检索过滤——但它匹配的是**形式**，不是含义。

永远用原始字符串

```
import re
re.search(r"\d+", "id 42")      # r"..." = 原始字符串
```

不带 `r"..."`，Python 会吃掉反斜杠（`\d` → 报错/乱码）。**规则：**每个正则模式都写成原始字符串。

保命符号表

符号	匹配
<code>.</code>	任意字符（换行除外）
<code>\d \w \s</code>	数字 / 单词字符 / 空白
<code>\D \W \S</code>	以上的取反
<code>+ * ?</code>	1+、0+、0 或 1
<code>{m} {m,n}</code>	恰好 m 次 / m 到 n 次
<code>^ \$</code>	字符串开头 / 结尾
<code>[abc] [a-z] [^abc]</code>	集合内 / 范围内 / 集合外
<code>(...)</code>	捕获组
<code>a b</code>	a 或 b

. 的陷阱

```
re.search(r".", "a.b").group()    # 'a' — 任意字符，不是点！
re.search(r"\.", "a.b").group()   # '.' — 想要真正的点就转义
```

想按字面意思用特殊字符时要转义：`\. \^ \$ \* \+ \? \(\) \[ \] \{ \} \|`

你需要的四个函数

```
re.search(pattern, s)      # 任意位置的第一个匹配 -> 匹配对象或 None
re.fullmatch(pattern, s)   # 整个字符串必须匹配 -> 校验用
re.findall(pattern, s)     # 所有匹配的列表
re.sub(pattern, repl, s)   # 替换匹配 -> 清洗用
```

大小写不敏感用 `re.IGNORECASE`：`re.search(p, s, re.IGNORECASE)`。

校验（fullmatch 锁住两端）

```
def valid_university_email(addr):
    return re.fullmatch(r"\w+@\w+\.edu", addr) is not None

valid_university_email("ana@university.edu") # True
valid_university_email("ana@gmail.com")      # False
valid_university_email("ana@x.edu")          # False
```

用捕获组提取

```
m = re.search(r"([A-Z]{2})(\d{4})", "Course ED1234 meets Tue")
m.group(0)    # "ED1234" 完整匹配
m.group(1)    # "ED"      系别
m.group(2)    # "1234"    课号
```

`m.groups()` 一次交出所有捕获：`('ED', '1234')`。没匹配到时 `m` 是 `None`——调 `.group()` 之前先检查。

清洗与挖掘自由文本

```
re.sub(r"\s+", " ", messy).strip()      # 折叠空白
re.findall(r"#(\w+)", "love #python #stats") # ['python', 'stats']

from collections import Counter
Counter(re.findall(r"#(\w+)", corpus))    # 主题词频
```

用分组重排：`re.sub(r"^(.+),\s*(.+)$", r"\2 \1", "Curie, Marie")` → `"Marie Curie"`。

什么时候不该用正则

```
"a,b,c".split(",")      # 简单切分 — 不需要正则
"  hi  ".strip()        # 去空白 — 不需要正则
text.replace("X", "Y")   # 固定子串 — 不需要正则
url.removeprefix("https://") # 去掉已知前缀 — 不需要正则 (3.9+)
```

正则擅长 *可变* 的模式。固定字符串用普通方法更好读。

轮到你了

`examples/session-09/practice.md`：

1. 邮箱校验器。
2. 提取系别+课号。
3. 折叠空白。
2. 统计各回答中的 #话题标签。
5. 翻转 `"Last, First"`。
6. 一个该用 `.split()` 的场景。

更进一步

正则的火力升级

标志位

```
re.findall(r"stress", corpus, re.IGNORECASE) # Stress、STRESS、stress
re.findall(r"^\d+", text, re.MULTILINE)      # ^ 和 $ 按行匹配
re.search(p, s, re.IGNORECASE | re.MULTILINE) # 用 | 组合
re.search(r"a.b", text, re.DOTALL)           # . 也能跨过换行
```

`re.compile` —— 给模式起名字

```
EMAIL = re.compile(r"\w+@\w+\.edu")
COURSE = re.compile(r"[A-Z]{2}\d{4}")

EMAIL.fullmatch(addr)      # 同样四个函数，现在是方法
COURSE.findall(text)
```

编译一次，处处复用——模式有了一个读者可以信任的 *名字*。

可读的模式：`re.VERBOSE`

```
EMAIL = re.compile(r"""
    \w+          # 用户名部分
    @
    \w+          # 域名
    \.edu        # 一个真正的点，然后是 edu
""", re.VERBOSE)
```

空白被忽略、允许注释——未来的自己也能审阅的正则。

分组，进阶版

```
m = re.search(r"(?P<dept>[A-Z]{2})(?P<num>\d{4})", "ED1234")
m.group("dept"), m.groupdict()      # 名字胜过序号

r"(?:Prof|Dr)\.?\s+(\w+)"          # (?:...) 只分组、不捕获
```

要提取的组起个名字；只为分组的括号用 `(?:...)`。

贪婪 vs 惰性

```
re.search(r"\[(.+)\]", "[ED101][ED102]").group(1)  # 'ED101][ED102' ❷
re.search(r"\[(.+?)\]", "[ED101][ED102]").group(1) # 'ED101'
```

`+` 和 `*` 能吞多少吞多少；`+` / `*` 能少吞就少吞。括号里的、引号里的内容，几乎总是想要惰性。

`re.sub` 传入函数：匿名化器

```
ids = {}
def anonymize(m):
    name = m.group(0)
    ids.setdefault(name, f"P{len(ids) + 1:03d}")
    return ids[name]

re.sub(r"\b(?:Ana|Ben|Cara)\b", anonymize, transcript)
# "P001 said ... P002 replied ... P001 agreed"
```

替换内容需要计算时，传一个函数——每个匹配都会经过它。（对 `ids` 的闭包——第 5 课的回报。）

正则遇上文件

```
text = Path("responses.txt").read_text(encoding="utf-8")
emails = sorted(set(EMAIL.findall(text)))
```

第 8 课 + 第 9 课，两行搞定：读文件、挖模式、用集合去重。

轮到你了——第二轮

`examples/session-09/practice.md` → **In class — going deeper**：带注释的 `VERBOSE` 邮箱模式、匿名化器，以及双格式日期收割。

陷阱回顾

- `.` 匹配任意字符——真正的点用 `\.`。
- 忘写 `r"..."` 会毁掉你的反斜杠。
- `re.search` 没匹配到返回 `None`——先判断再 `.group()`。
- 字符串方法更清晰的地方，别上正则。

小结

你已经能校验、提取和清洗真实世界的文本。 **下一课**：第 10 课——模块、面向对象与 Python 惯用法。

课后作业（第 10 课之前）

课外完成——不计入课堂时间。完整题目 + 参考答案：`examples/session-09/practice.md` → *Homework*。

1. **模式练习** —— 学号、美式电话、ISO 日期：各写一个 `fullmatch` 模式。
2. **脏姓名清洗** —— 用 `re.sub` 规范一系列爬来的姓名。
3. **域名收割** —— 从一段文字里提取所有邮箱域名。

练习与作业

课堂练习

永远用原始字符串 `r"..."`。每个结果先预测再运行。

任务 1 —— 校验

写 `valid_university_email(addr)`，只对 `something@something.edu` 返回 `True`。测试：`"ana@university.edu"`、`"ana@gmail.com"`、`"a@b.edu.evill.com"`。

任务 2 —— 用分组提取

从 "Course ED1234 meets Tue" 中用一个带两个捕获组的正则取出系别 (ED) 和 课号 (1234)。

任务 3 —— 清洗

把 " too much\t space " 里成串的空白折叠成单个空格并去掉首尾。

任务 4 —— 挖掘自由文本

统计一组开放式回答里每个 #话题标签 出现的次数 (用 `re.findall(r"#(\w+)", text)` 和 `collections.Counter`)。

任务 5 —— 重排

用一个正则 + 分组把 "Curie, Marie" 变成 "Marie Curie"。

任务 6 —— 判断力

举一个用普通字符串方法 (`.split()`、`.strip()`、`.replace()`) 比正则更好、更清楚的任务。

加餐 —— Python 惯用法速练

盖住 # -> 后面的答案，逐行预测，再运行。

```
import re
m = re.search(r"(?P<year>\d{4})", "class of 2026")
print(m.group("year"), m.groupdict()) # -> 2026 {'year': '2026'} (命名分组)
print(re.split(r"\s*,\s*", "a, b ,c")) # -> ['a', 'b', 'c'] (按逗号带空格切分)
```

课堂练习——更进一步 (第二小时)

E1 —— 命名分组

用 `(?P<dept>...)` / `(?P<num>...)` 重做系别+课号提取，打印 `m.groupdict()`。

E2 —— regex101 实地考察

把你的邮箱模式贴进 regex101.com (flavor 选 **Python**)。逐符号读左栏的解释——它说的是你想表达的意思吗？

D1 —— 带注释的模式

用 `re.compile(..., re.VERBOSE)` 重写邮箱校验器，每个符号一行注释。行为必须完全一致。

D2 —— 匿名化器

用 `re.sub` 传函数替换，把转录文本里出现的 Ana/Ben/Cara 全部替换成稳定编号 P001、P002 (同名 → 同编号)。

D3 —— 双格式日期收割

从 "submitted 2026-07-06, revised 07/08/2026, due 2026-09-01" 里用一次 `findall` + 一个候选分支，提取全部日期（`YYYY-MM-DD` 或 `MM/DD/YYYY` 两种格式）。

D4 —— 忽略大小写的关键词计数

用标志位统计一段话里 "stress" 的出现次数、大小写不限——不许先把文本转小写。

课后作业（第 10 课之前）

约 30–45 分钟，课外完成——不计入课堂时间。先都试一遍，再看答案。

H1 —— 模式练习

各写一个 `re.fullmatch` 模式；每个用两个合法、两个非法的字符串测试：

1. 学号：两个大写字母 + 六位数字（`AB123456`）
2. 美式电话：`555-867-5309`（只有数字和连字符）
3. ISO 日期：`2026-07-06`（只验形状——不用检查月份范围）

H2 —— 脏姓名清洗

把 `["smith, ana", "LEE,BEN", "Garcia , Cara "]` 规范成 `"Ana Smith"`、`"Ben Lee"`、`"Cara Garcia"`：按逗号（两侧可有空格）做正则切分，然后 `.strip()` + `.title()`，再交换顺序。

H3 —— 域名收割

从含有多个邮箱地址的一段文字里，用一次 `findall` + 一个 `set` 提取不重复的域名（如 `{"university.edu", "gmail.com"}`）。

参考答案

课堂练习

见本文件夹的 `demo.py` ——六个任务的完整实现。关键行：

```
re.fullmatch(r"\w+@\w+\.edu", addr) is not None      # 1 (fullmatch 锁住两端)
m = re.search(r"([A-Z]{2})(\d{4})", s); m.group(1), m.group(2)  # 2
re.sub(r"\s+", " ", messy).strip()                  # 3
from collections import Counter; Counter(re.findall(r"#(\w+)", text))  # 4
m = re.search(r"^(.+),\s*(.+)$", s); f"{m.group(2)} {m.group(1)}"    # 5
```

任务 6：按逗号切 `"a,b,c"` 就是 `"a,b,c".split(",")` ——不需要正则。只有模式真正可变（数字、可选部分、锚点）时才请出正则。

陷阱提醒：`.` 匹配任意字符——真正的点用 `\.`；永远别忘 `r"..."` 前缀，否则反斜杠会变成 Python 的转义序列。

课堂练习——更进一步

```
# E1
import re
m = re.search(r"(?P<dept>[A-Z]{2})(?P<num>\d{4})", "Course ED1234 meets Tue")
print(m.group("dept"), m.group("num"))    # ED 1234
print(m.groupdict())                     # {'dept': 'ED', 'num': '1234'}
```

E2 —— 重点是习惯：regex101 的解释器能在你的数据咬你之前，先抓住 "." 匹配任意字符"这类失误。

```
import re

# D1
EMAIL = re.compile(r"""
    \w+          # 用户名部分
    @
    \w+          # 域名
    \.edu        # 一个真正的点，然后是 edu
""", re.VERBOSE)
print(EMAIL.fullmatch("ana@university.edu") is not None)    # True

# D2
ids = {}
def anonymize(m):
    name = m.group(0)
    ids.setdefault(name, f"P{len(ids) + 1:03d}")
    return ids[name]

text = "Ana said X. Ben said Y. Ana agreed."
print(re.sub(r"\b(?:Ana|Ben|Cara)\b", anonymize, text))
# P001 said X. P002 said Y. P001 agreed.

# D3
s = "submitted 2026-07-06, revised 07/08/2026, due 2026-09-01"
print(re.findall(r"\d{4}-\d{2}-\d{2}|\d{2}/\d{2}/\d{4}", s))
# ['2026-07-06', '07/08/2026', '2026-09-01']

# D4
para = "Stress was high. Some stress is normal. STRESS!"
print(len(re.findall(r"stress", para, re.IGNORECASE)))    # 3
```

课后作业

```
# H1
import re

sid    = r"[A-Z]{2}\d{6}"          # AB123456 ✓   CD000001 ✓   ab123456 ✗   AB12345 ✗
phone  = r"\d{3}-\d{3}-\d{4}"      # 555-867-5309 ✓   555-8675309 ✗
date   = r"\d{4}-\d{2}-\d{2}"      # 2026-07-06 ✓   2026-7-6 ✗

for s in ["AB123456", "ab123456"]:
    print(s, re.fullmatch(sid, s) is not None)

# H2
def clean_name(raw):
    last, first = re.split(r"\s*,\s*", raw.strip(), maxsplit=1)
    return f"{first.strip().title()} {last.strip().title()}"

for raw in [" smith, ana", "LEE,BEN", "Garcia , Cara "]:
    print(clean_name(raw))          # Ana Smith · Ben Lee · Cara Garcia

# H3
text = "Write ana@university.edu or ben@gmail.com; cc cara@university.edu today."
print(set(re.findall(r"\w+@([\w.]+\.\w+)", text)))
# {'university.edu', 'gmail.com'} — 集合顺手去了重
```

陷阱 —— 先预测，再核对

盖住结果，逐个预测，再自己核对。

1.

```
import re
m = re.match(r'\[(.+)\]', '[ED101][ED102]')
m.group(1)
```

结果: `'ED101][ED102'`。 `+` 是贪婪的——能抓多少抓多少。想要最短匹配用 `+` : `r'\[(.+?)\]'`。

2.

```
import re
bool(re.match(r'\d+', 'cohort2026'))
```

结果: `False`。 `re.match` 锚定在字符串**开头**。想在任意位置找匹配用 `re.search`。

3.

```
import re
re.fullmatch(r'grades.csv', 'gradesXcsv') is not None
```

结果: `True`。 `.` 匹配**任意字符**，所以 `gradesXcsv` 也符合模式。想要真正的点就转义——`r'grades\.csv'`。

第 10 课

模块、面向对象与 Python 惯用法

模块：把代码拆进多个文件

```
# grades.py
def letter_grade(score): ...
def class_average(scores): ...

# analysis.py
from grades import letter_grade, class_average
```

一个 `.py` 文件就是一个**模块**。`import` 让它的函数在别处复用 → 不再复制粘贴。

`__main__` 守卫

```
# grades.py
if __name__ == "__main__":
    print(letter_grade(85)) # 只在直接执行 grades.py 时运行
```

被 `import grades` 时，`__name__` 是 `"grades"`，这块代码就被跳过。一个文件既能当脚本运行，又能当库导入。

OOP：为领域实体建模

```
class Student:
    def __init__(self, name, gpa): # 构造器
        self.name = name
        self.gpa = gpa
    def __str__(self): # 打印时的样子
        return f"{self.name} ({self.gpa})"

ana = Student("Ana", 3.9)
print(ana) # Ana (3.9)
```

□ 类就是一份**操作性定义**：哪些属性和行为“算”一个 Student。`self` = “眼下这一个学生”。

@property：赋值时校验

```
class Student:
    ...
    @property
    def gpa(self):
        return self._gpa
    @gpa.setter
    def gpa(self, value):
        if not 0 <= value <= 4:
            raise ValueError("gpa must be 0-4")
        self._gpa = value
```

现在 `ana.gpa = 5.0` 会抛错——对象捍卫自己的完整性。（`_gpa` 的前导下划线是“内部实现，请勿触碰”的约定；Python 选择信任你，而不是强制。）

继承

```
class GradStudent(Student):
    def __init__(self, name, gpa, advisor):
        super().__init__(name, gpa)    # 复用父类的初始化
        self.advisor = advisor
    def __str__(self):
        return super().__str__() + f" - {self.advisor}"
```

`GradStudent` 是一种 `Student`，再加点料。`super()` 调用父类。

Python 惯用法工具箱（巡礼）

```
[s.name for s in roster if s.gpa >= 2.0]    # 推导式（第 4 课）
list(map(lambda s: s.name.upper(), roster))# map：对每个都应用
list(filter(lambda s: s.gpa < 2.0, roster))# filter：留下符合的
for i, s in enumerate(roster): ...          # 序号 + 元素
for a, b in zip(names, scores): ...         # 并排走
```

生成器与海象运算符

```
def gpas(students):
    for s in students:
        yield s.gpa          # 一次产出一个 — 大数据也不占内存

g = gpas(roster)
list(g)    # ?
list(g)    # ? ← 跑两次—有什么变化？（见下方陷阱区）

if (n := len(roster)) > 30:   # 海象 := : 赋值 + 判断一步完成
    print(f"{n} students")
```

轮到你了

`examples/session-10/practice.md` :

1. 从 `grades.py` 导入。2. 写带校验的 `Student` 类。
2. 加上用 `super()` 的 `GradStudent`。4. 推导式 + `map` + `filter` + 一个生成器。

更进一步

用对象做设计

`__repr__` 和它的伙伴们

```
class Student:
    def __repr__(self):          # 给开发者看 (REPL、列表、日志)
        return f"Student({self.name!r}, {self.gpa})"
    def __eq__(self, other):
        return (self.name, self.gpa) == (other.name, other.gpa)
    def __lt__(self, other):     # "小于" — sorted() 直接可用
        return self.gpa < other.gpa

sorted(roster)                  # 不再需要 key=
```

双下方法让你的对象举止如内置类型：可打印、可比较、可排序——甚至可相加：定义 `__add__(self, other)`，`total = quiz + final` 就能跑。所谓“运算符重载”，仅此而已。

@dataclass , 进阶版

```
from dataclasses import dataclass, field

@dataclass
class Student:
    name: str
    gpa: float = 0.0
    courses: list = field(default_factory=list)    # 千万别写 courses: list = []
```

`default_factory=list` 就是第 5 课"可变默认值"法则的类版本——每个实例 一个新列表。

`@dataclass(frozen=True)` 给你一条不可变记录。

@classmethod : 备选构造器

```
@dataclass
class Student:
    name: str
    score: int

    @classmethod
    def from_row(cls, row):          # CSV 行(第 8 课) -> 对象
        return cls(row["name"], int(row["score"]))

roster = [Student.from_row(r) for r in csv.DictReader(f)]
```

`cls` 就是类本身。套路：**文件格式挡在边缘，程序内部全是对象。**（`@staticmethod` 是第三种口味：住在类命名空间里的普通函数——没有 `self`，也没有 `cls`。）

组合优先于继承

```
class Cohort:                          # Cohort 拥有 Student (组合 ✓)
    def __init__(self, name):
        self.students = []

class GradStudent(Student):            # GradStudent 是一种 Student (继承 ✓)
    ...
```

只有真正的"是一种"关系才继承。拿不准时，让对象持有对象——更灵活，也更好测试。

类属性 vs 实例属性——铁律

```
class Course:
    school = "Ed School"          # 类属性：所有实例共享 — 只放常量
    def __init__(self):
        self.students = []       # 实例属性：每个对象自己一份
```

可变数据永远放进 `__init__`。（你踩过那个共享列表陷阱——这就是它教的法则。）想用 `global` 的时候：能活过一次函数调用的状态，应该住在对象上。

生成器管道

```
def to_ints(rows):
    for r in rows:
        try:
            yield int(r)
        except ValueError:
            pass

def in_range(vals, lo=1, hi=5):
    yield from (v for v in vals if lo <= v <= 5)

clean = list(in_range(to_ints(raw)))    # 消费之前，什么都不会跑
```

一级产出交给下一级——**惰性**、常数内存，百万行的文件也照吃。（水面之下，每个 `for` 说的都是同一套协议：`iter()` 拿到迭代器，`next()` 取到空为止。生成器就是天生会说这门话的对象。）

从脚本到项目

```
survey_tool/
├─ clean.py      # 校验函数
├─ stats.py      # 汇总统计
├─ report.py     # 主入口：python3 report.py
└─ tests/test_clean.py
```

一个模块管一件事；`report.py` 导入其余部分；测试放旁边。`report.py` 里的惯例：

```
def main():
    ...                                # 程序的正事，集中一处

if __name__ == "__main__":
    main()
```

每一种原料你都已经认识——这只是摆盘。（若 `pytest` 找不到 `tests/`，在里面放一个空的 `__init__.py`，把它标记为包。）

轮到你了——第二轮

`examples/session-10/practice.md` → **In class — going deeper**：可排序的 `Student`、带 `default_factory` 的 `dataclass`、`from_row`，以及一条两级生成器管道。

陷阱回顾

- `self` 就是“这一个实例”——没有魔法。
- 生成器只能遍历一次，之后就空了。
- `__main__` 守卫防止被导入的模块跑演示代码。
- 函数或字典能解决的，别硬上类。

小结

你已经能把代码组织成模块和类，写出地道的 Python。下一步（可选）：毕业项目——把一切串起来。

课后作业（毕业项目之前）

课外完成——不计入课堂时间。完整题目 + 参考答案：`examples/session-10/practice.md` → *Homework*。

1. `Student.courses` —— 课程列表 + 计算出的 `gpa` —— 数据和守护它的规则在一起。
2. `Cohort` 类 —— 装着一群 `Student`；`mean_gpa()`、`at_risk()`。
3. **Pythonic 重写** —— 三个给定循环 → 推导式、生成器、`map`。

练习与作业

课堂练习

本文件夹的文件：`grades.py`（供你导入的模块）、`demo.py`（成品示例）。

任务 1 —— 使用模块

在新文件里 `from grades import letter_grade, class_average` 并调用两者。为什么导入时 `grades.py` 里的 `if __name__ == "__main__":` 块不会运行？

任务 2 —— 带校验属性的类

写 `Student(name, gpa)`：

- `__str__` → `"Ana: 3.9 (Good)"`，
- `standing()` → `gpa ≥ 2.0` 时 `"Good"` 否则 `"Probation"`，
- `gpa` 的 `@property` setter：超出 0–4 抛 `ValueError`。证明 setter 会拒绝 `5.0`。

任务 3 —— 继承

加 `GradStudent(Student)`：多存一个 `advisor`，用 `super().__init__(...)`；重写 `__str__` 把导师名附在后面。

任务 4 —— Python 惯用法工具箱

给定一组 `Student`：

1. 状态良好者的姓名（列表推导式），
2. 大写的姓名（`map`），
3. 有风险的学生（`filter`），
4. 用一个 `yield` 每个 gpa 的生成器求平均 gpa——然后展示第二次遍历时它已空。

加餐 —— Python 惯用法速练

盖住 `# ->` 后面的答案，逐行预测，再运行。

```
from dataclasses import dataclass

@dataclass                                # 自动 __init__、__repr__、__eq__
class Point:
    x: int
    y: int
print(Point(1, 2), Point(1, 2) == Point(1, 2))    # -> Point(x=1, y=2) True

def head(v):
    match v:                                # 结构化模式匹配 (3.10+)
        case [first, *_]: return first
        case _: return None
print(head([9, 8]), head(5))                  # -> 9 None
```

课堂练习——更进一步（第二小时）

E1 —— 亲眼看守卫

在 `grades.py` 顶部加 `print("running as", __name__)`。先直接运行文件，再在 REPL 里 `import grades`——各打印什么？为什么？

E2 —— `@dataclass` 版 `Student`

把普通（不带校验的）`Student` 改写成 `@dataclass`。白得了什么？在两个等值学生上检查 `repr` 和 `==`。

D1 —— 可排序的学生

给 `Student` 写 `__repr__`、`__eq__` 和按 gpa 的 `__lt__`，让 `sorted(roster)` 不用 `key=` 就能跑。证明它。

D2 —— 正确的 `dataclass`

写 `@dataclass class Course: name: str; roster: list = field(default_factory=list)`。
`roster: list = []` 为什么是错的——这是第 5 课哪个 bug 的类版本？

D3 —— `from_row`

给 `Student` 加 `from_row(cls, row)`：从 CSV 的 `DictReader` 行构造实例（记得转换类型！）。为什么 `@classmethod` 是它的正确归宿？

D4 —— 两级管道

写生成器 `to_ints(cells)`（跳过转不动的）和 `in_range(vals, lo=1, hi=5)`；串起来处理 `["5", "3", "N/A", "7", "1"]`。真正的计算发生在什么时候？

课后作业（毕业项目之前）

约 30–45 分钟，课外完成——不计入课堂时间。先都试一遍，再看答案。

H1 —— 课程与计算出的 GPA

扩展 `Student`：一个（课程名，绩点）元组的 `courses` 列表、一个校验 0–4 的 `add_course(name, points)` 方法，并把 `gpa` 做成**计算属性**（课程绩点的均值；没有课程时为 `None`）。不存任何会过期的 `gpa`。

H2 —— `Cohort` 类

`Cohort(name)` 装着一群 `Student`：`add(student)`、`mean_gpa()`（跳过没有 `gpa` 的学生）、`at_risk(threshold=2.0)` 返回姓名，`__str__` → `"Fall-26 (3 students)"`。想清楚学生列表放哪——记得类变量陷阱。

H3 —— Pythonic 重写

把每段改写成一行推导式 / 生成器 / `map`：

```
# 1 -> 列表推导式
out = []
for s in roster:
    if s.gpa is not None and s.gpa >= 3.5:
        out.append(s.name)

# 2 -> sum() 里的生成器（不建列表）
total = 0
for s in roster:
    total += len(s.courses)

# 3 -> map（或推导式——说说你选哪个、为什么）
labels = []
for s in roster:
    labels.append(str(s))
```

参考答案

课堂练习

见 `demo.py` ——任务 2–4 的完整实现。要点：

```
@property
def gpa(self): return self._gpa
@gpa.setter
def gpa(self, v):
    if not 0 <= v <= 4: raise ValueError(...)
    self._gpa = v

class GradStudent(Student):
    def __init__(self, name, gpa, advisor):
        super().__init__(name, gpa)
        self.advisor = advisor

[s.name for s in roster if s.gpa >= 2.0]          # 推导式
list(map(lambda s: s.name.upper(), roster))      # map
[s.name for s in filter(lambda s: s.gpa < 2.0, roster)] # filter
def gpas(rs):
    for s in rs: yield s.gpa                     # 生成器（遍历一次即耗尽）
```

任务 1： `__name__` 只有在文件被**直接运行**时才是 `"__main__"`；被 `import` 时它是 `"grades"`，那段演示代码被跳过——这就是一个文件既能当脚本又能当库的原理。

课堂练习——更进一步

```
# E1
# python3 grades.py -> "running as __main__" (这个文件就是程序本体)
# >>> import grades -> "running as grades" (于是 __main__ 块被跳过)

# E2
from dataclasses import dataclass

@dataclass
class Student:
    name: str
    gpa: float

# 白得的: __init__、好读的 __repr__、逐字段的 __eq__:
print(Student("Ana", 3.9) == Student("Ana", 3.9)) # True

# D1
class Student:
    def __init__(self, name, gpa):
        self.name = name
        self.gpa = gpa
    def __repr__(self):
        return f"Student({self.name!r}, {self.gpa})"
    def __eq__(self, other):
        return (self.name, self.gpa) == (other.name, other.gpa)
    def __lt__(self, other):
        return self.gpa < other.gpa

roster = [Student("Ana", 3.9), Student("Ben", 1.8), Student("Cara", 3.2)]
print(sorted(roster)) # Ben、Cara、Ana — __lt__ 教会了 sorted() 怎么排

# D2
from dataclasses import dataclass, field

@dataclass
class Course:
    name: str
    roster: list = field(default_factory=list)
# `roster: list = []` 会让所有 Course 共用同一个列表 — 第 5 课的可变默认值
# bug 换了件类的外衣。default_factory 给每个实例现造一个新列表。

# D3
@dataclass
class Student2:
    name: str
    score: int

    @classmethod
    def from_row(cls, row):
        return cls(row["name"], int(row["score"]))
# 它构造的是类本身的实例，所以属于类 (cls) 而不属于任何一个对象—
# "备选构造器"模式。

# D4
def to_ints(cells):
    for c in cells:
```

```
        try:
            yield int(c)
        except ValueError:
            pass

def in_range(vals, lo=1, hi=5):
    yield from (v for v in vals if lo <= v <= hi)

pipeline = in_range(to_ints(["5", "3", "N/A", "7", "1"]))
print(list(pipeline))    # [5, 3, 1] — 直到 list() 把值拉过来才真正运行
```


课后作业

```
# H1
class Student:
    def __init__(self, name):
        self.name = name
        self.courses = []  # (课程, 绩点) 元组

    def add_course(self, course, points):
        if not 0 <= points <= 4:
            raise ValueError(f"{points} outside 0-4")
        self.courses.append((course, points))

    @property
    def gpa(self):  # 计算出来的 — 永不过期
        if not self.courses:
            return None
        return sum(p for _, p in self.courses) / len(self.courses)

    def __str__(self):
        return f"{self.name} (GPA {self.gpa})"

# H2
class Cohort:
    def __init__(self, name):
        self.name = name
        self.students = []  # 放在实例上 — 千万不是类变量！

    def add(self, student):
        self.students.append(student)

    def mean_gpa(self):
        gpas = [s.gpa for s in self.students if s.gpa is not None]
        return sum(gpas) / len(gpas) if gpas else None

    def at_risk(self, threshold=2.0):
        return [s.name for s in self.students
                if s.gpa is not None and s.gpa < threshold]

    def __str__(self):
        return f"{self.name} ({len(self.students)} students)"

# H3
names = [s.name for s in roster if s.gpa is not None and s.gpa >= 3.5]
total = sum(len(s.courses) for s in roster)  # 生成器 — 内存里不建列表
labels = list(map(str, roster))  # 或 [str(s) for s in roster] — 都行；
# 多数人觉得推导式更好读
```

陷阱 —— 先预测，再核对

盖住结果，逐个预测，再自己核对。

1.

```
gpas = (s for s in [3.9, 1.8, 3.2])
first_pass = list(gpas)
list(gpas)
```

结果: `[]`。生成器是一次性的：完整遍历一遍后就耗尽了。要么重建，要么需要用两次就先存成列表。

2.

```
from dataclasses import dataclass

@dataclass
class Grade:
    course: str
    score: int

g1 = Grade('ED101', 91)
g2 = Grade('ED101', 91)
g1 == g2
```

结果: `True`。 `@dataclass` 自动写了逐字段比较的 `__eq__`，所以值相同的成绩 `==` 成立（尽管 `g1 is g2` 为 `False`）。

3.

```
class Course:
    students = []
    def enroll(self, name):
        self.students.append(name)

art = Course()
math = Course()
art.enroll('Ana')
math.students
```

结果: `['Ana']`。 `students` 是所有实例共享的类变量。在 `__init__` 里给每个实例自己的：
`self.students = []`。

环境配置与工具

在自己电脑上运行 Python 所需的一切——外加让学习提速的工具（和 AI 使用习惯）。

最快起步：什么都不装

本站每个示例都能在浏览器里运行——直接按 **Run** 就行。无需安装，前几课用它正合适。想处理自己的文件时，再按下面的步骤本地安装 Python。

用 Jupyter 笔记本学整门课

每一课同时也是一个 **Jupyter 笔记本**——同样的示例和练习，做成可以逐格运行的文档。三种用法，选顺手的：

- **浏览器里、免安装** —— 打开在线 JupyterLite 工作台：<https://yangnei.github.io/learn-python/jupyter/lab/index.html>。是真正的 Jupyter，完全在你的浏览器里运行；改动保存在本地浏览器中，什么都不会上传。每课页面也有直接打开该课笔记本的按钮。
- **Google Colab** —— 每课页面有 **Open in Colab** 按钮（免费，跑在 Google 云端，需要 Google 账号）。想存到 Drive 或做更重的数据工作时很方便。
- **本地 Jupyter** —— 装好 Python 后（见下）：`pip install jupyterlab`，运行 `jupyter lab`，打开你从课程页面下载的 `.ipynb`。

笔记本怎么开：点一个单元格，按 **Shift + Enter** 运行并跳到下一格。运行前先预测输出——惊讶就是课程本身——然后改一改、再运行，做实验。

自动补全与装库：按 **Tab** 补全名字，**Shift + Tab** 弹出函数帮助。要加包，在单元格里运行 `%pip install <名字>`（如 `%pip install pandas`）。浏览器版（JupyterLite）只支持兼容 Pyodide 的包——`pandas`、`numpy`、`matplotlib` 这些大件都行——安装只在本次会话有效，内核重启后重跑那一格。本地 Jupyter 或 Colab 里，`%pip install` 对该环境是永久的。

安装 Python（按你的系统选）

Windows

- 官方安装包：<https://www.python.org/downloads/windows/> —— 第一屏勾选 **"Add python.exe to PATH"**，再点 *Install Now*。
- （备选）Microsoft Store → 搜索 **Python 3.12**。
- 验证成功——打开 **PowerShell** 运行：`python --version`
- 官方指南：<https://docs.python.org/3/using/windows.html>

macOS

- 官方安装包：<https://www.python.org/downloads/macos/>
- (备选) Homebrew：`brew install python`
- 验证成功——打开终端运行：`python3 --version`
- 官方指南：<https://docs.python.org/3/using/mac.html>

Linux

- 通常已预装。若没有：`sudo apt install python3 python3-pip` (Debian/Ubuntu) 或 `sudo dnf install python3` (Fedora)。
- 验证成功：`python3 --version`
- 官方指南：<https://docs.python.org/3/using/unix.html>

编辑器：安装 VS Code (<https://code.visualstudio.com/>) 和微软出品的 **Python 扩展**。图文教程：<https://code.visualstudio.com/docs/python/python-tutorial>。想要更轻量的？Thonny (<https://thonny.org/>) 是自带变量查看器和调试器的新手 IDE。

运行脚本：代码存成 `name.py`，然后在终端运行 `python name.py` (Windows) 或 `python3 name.py` (macOS/Linux)。

装第三方包 (比如第 8 课的 pandas 预告)：`pip install pandas` (或 `pip3`)。

亲眼看代码怎么跑 —— Python Tutor

<https://pythontutor.com/> —— 贴上代码，**逐行单步执行**，实时看到变量、调用栈，以及哪个名字指着哪个对象。对本课程反复告诫的那些坑，它是最好的可视化工具：

- **别名** —— 亲眼看 `b = a` 让两个名字指向同一个列表 (第 4 课)，
- **可变默认参数** bug (第 5 课)，
- **递归调用栈** 的堆起与展开 (第 6 课)。

每当你想“等等，这为什么会这样？”——把代码片段丢进 Python Tutor。

用 AI 学习 (但别让它替你干活)

AI 助手 (Claude 等) 是巨大的加速器——*前提是让它解释，而不是让它写*。每个答案都要自己敲。适合本课程的提示词：

- “给初学者总结这页文档，只要我需要的 5 件事：\<贴文本或 URL>” —— 把密不透风的参考页变成能用的东西。
- “解释这个报错和最可能的原因，指出是哪一行：\<贴回溯信息>”
- “先预测这段代码打印什么，再解释为什么：\<代码>” —— 然后自己运行、对答案。这种先预测再运行的习惯就是本课程的方法论。

- "按本课程的陷阱清单审查我的函数——同一 vs 相等、别名、可变默认值、差一错误——但不要替我重写。"
- "给我 3 道更难的同类练习，答案先藏起来。"

经验法则：让它解释，别让它代劳。如果它甩给你一段你读不懂每一行的代码，让它慢下来逐行讲。

其他称手工具

- **regex101.com** (<https://regex101.com/>) —— 交互式构建并解释正则表达式，带实时匹配视图（第 9 课）。记得把 flavor 设为 **Python**。
- **官方文档** —— [Python 教程](#)和 [标准库参考](#)。都收藏好。
- **Google Colab** (<https://colab.research.google.com/>) —— 浏览器里跑 Python 笔记本，免安装；开始做真实数据工作后的自然一步。
- 本站的[陷阱与坑速查表](#)——整门课都开着它。

Python 陷阱与坑 —— 主速查表

咬新手（也咬不少老手）的语言怪癖。以下每个行为都在 CPython 3.11+ 上实际运行并验证过。看惊讶一栏、盖住原因/修法，先预测。整门课程都把它开着。

1 — 相等 `==` vs 同一 `is` (第2课)

表达式	结果	
<code>a = [1,2]; b = [1,2]; a == b</code>	<code>True</code>	值相同
<code>a = [1,2]; b = [1,2]; a is b</code>	<code>False</code>	内存中是不同的对象
<code>a = [1,2]; b = a; a is b</code>	<code>True</code>	<code>b</code> 是同一个对象（别名）

- `==` 问"值相同吗？"——几乎永远是你想要的。
 - `is` 问"是同一个对象吗？"——只用于 `None`、`True`、`False`：`python if x is None: # ✓ 正确`
`if x == None: # △ 能跑，但不地道 — 用 `is``
 - 身份的小怪癖（千万别依赖）：CPython 缓存小整数（-5 到 256）并驻留部分字符串，所以 `a = 256; b = 256; a is b` 为 `True`，到 257 就可能是 `False`。这是实现细节。比值，永远用 `==`。
- 桥接：GPA 相同的两名学生是 `==`；同一名学生才是 `is`。

2 — 布尔值就是整数 (第2课)

```
True == 1           # True    (bool 是 int 的子类)
False == 0          # True
5 + True            # 6       对，布尔值能做算术
sum([True, False, True]) # 2   ← 数出 True 的个数
isinstance(True, int) # True
type(True) is int    # False   (它的类型是 bool，一个子类型)
```

- 善用它：`sum(conditions)` 是数"多少个为真"的 Python 惯用法。
 - 陷阱：尽管 `True == 1`，`True is 1` 却是 `False`（不同对象）。用 `==` 比较。
- 桥接：这就是语言内置的哑变量编码（1/0）——对标志求和就是在数个案。

3 — 浮点精度 (第 2 课)

```
0.1 + 0.2          # 0.30000000000000004
0.1 + 0.2 == 0.3    # False
```

- **原因**：小数以二进制存储；大多数小数无法精确表示。不是 Python 的 bug——每门语言都这样。
 - **修法**：`python import math math.isclose(0.1 + 0.2, 0.3) # True` ← 用容差比较浮点数
`round(0.1 + 0.2, 2) == 0.3 # True` ← 或者取整后展示/比较 `from decimal import Decimal # 真正需要时用精确十进制（金钱、成绩）` `Decimal("0.1") + Decimal("0.2") # Decimal('0.3')`
 - **法则**：永远不要用 `==` 检验计算出的浮点数。用 `math.isclose`（或 `round`）。
- **桥接**：你本来就从不对两个测量分数做精确相等判断——直觉相同，原因换了（二进制存储，不是测量误差）。

4 — 跨类型比较 (第 2 课)

```
5 == "5"          # False    (类型不同 → 不相等, 但不报错)
5 == 5.0           # True     (int 与 float 按数值比较)
5 > "5"            # ❌ TypeError: '>' not supported between 'int' and 'str'
```

- 不相干类型之间的**相等** (`== / !=`) → 直接 `False`，从不崩溃。
 - 不兼容类型之间的**排序** (`<`、`>`、`<=`、`>=`) → `TypeError`。
 - **修法**：先转换。`int("5") == 5 → True`。排序前用 `isinstance` 把关。
- **桥接**：计算机能跨类型回答“是不是一样？”，却无法给文本和数字**排名**——它们没有共同量尺。

5 — 序列逐元素比较 (第 2 / 4 课)

```
[1, 2] == [1, 2]    # True
[1, 2] == (1, 2)     # False  ← 列表 vs 元组：类型不同，永不相等
(1, 2) < (1, 3)      # True    ← 逐位置比较（字典序）
"apple" < "banana"   # True    ← 字符串逐字符比较（近似字母序）
[1, 2] < [1, 2, 3]   # True    ← 前缀算"小于"
```

- **陷阱**：内容完全相同的 `list` 和 `tuple` **永远不 `==`**——类型有一票否决权。
- 列表/元组/字符串从左到右比较；第一个不同的元素定胜负。

6 — 真值判断（什么算 False）（第 2 / 3 课）

```
# 这些都是"假值":
bool(0) bool(0.0) bool("") bool([]) bool({}) bool(set()) bool(None) # 全是 False
bool("0") # True! ← 非空字符串是真值, "0" 也不例外
bool([0]) # True ← 非空列表是真值, 哪怕装的是个 0
```

- **惯用法**：直接判断"空空空"—— `if scores:` 而不是 `if len(scores) > 0:` ; `if name:` 而不是 `if name != "":`。
- **陷阱**：字符串 `"0"` 和 `"False"` 都是真值。用户输入先转换再判断。

7 — `and` / `or` 返回操作数，不是布尔值（第 3 课）

```
5 and 0 # 0 (and → 第一个假值, 否则最后一个值)
0 or "hi" # "hi" (or → 第一个真值, 否则最后一个值)
"" or "N/A" # "N/A" ← 顺手的默认值惯用法
```

- **善用它**：`name = user_input or "Anonymous"` 提供默认值。
- **陷阱**：别写 `if x == True`，直接 `if x:`。另外 `and` / `or` 短路——右侧可能根本不运行，别把副作用藏在那里。

8 — 可变默认参数（第 5 课）—— 大名鼎鼎的那个

```
def add_student(name, roster=[]): # ✗ 危险
    roster.append(name)
    return roster

add_student("Ana") # ['Ana']
add_student("Ben") # ['Ana', 'Ben'] ☹️ 默认列表跨调用一直存在
```

- **原因**：默认的 `[]` 只在函数定义时创建一次，之后每次调用都复用它。
- **修法**：`python def add_student(name, roster=None): # ✓ if roster is None: roster = [] roster.append(name) return roster`
- **法则**：永远不要用可变默认值（`[]`、`{}`、`set()`）。用 `None`，在函数体内创建。

9 — 别名：变量是标签，不是盒子 (第 4 课)

```
a = [1, 2, 3]
b = a          # b 是同一个列表，不是副本
a.append(4)
b              # [1, 2, 3, 4] ☹️ b 也变了

b = a.copy()   # ✓ 现在 b 是独立的（浅）副本
import copy
b = copy.deepcopy(a)  # ✓ 连嵌套的列表/字典也独立
```

- **陷阱：** `grid = [[0] * 3] * 3` 造出同一行的三个引用——改 `grid[0][0]` 三行全变。要用 `[[0]*3 for _ in range(3)]`。
- **法则：** 赋值从不复制。 `=` 只是给同一个对象再绑一个名字。

□ **桥接：** 变量是贴在对象上的便利贴，不是装着值的容器。

10 — `type()` VS `isinstance()` (第 2 课)

```
isinstance(x, int)          # ✓ 地道；尊重继承
isinstance(x, (int, float)) # ✓ "x 是某种数字吗？"
type(x) is int              # 只认精确类型；很少是你想要的
type(x) == int              # 能跑，但类型身份请用 `is`
```

- **陷阱：** `isinstance(True, int)` 为 `True` (`bool` 是 `int` 的子类型)。必须排除布尔时，用 `type(x) is int` 或 `isinstance(x, int) and not isinstance(x, bool)`。

11 — 整数除法 vs 浮点除法 (第 1 / 2 课)

```
7 / 2      # 3.5    真除法永远返回浮点数
7 // 2     # 3      整除（向  $-\infty$  取整）
-7 // 2    # -4     ← 是向下取整，不是向零截断！
7 % 2      # 1      取余 — 用 % 2 判断奇偶
7 / 0      # ☐ ZeroDivisionError
```

- **陷阱：** `/` 即使结果是整数也给浮点数（`4 / 2` 是 `2.0`）。要整数结果用 `//`。

12 — 字符串不可变；有些方法返回而不修改 (第 1 / 9 课)

```
s = " Hello "
s.strip()      # "Hello" ← 返回一个新字符串
s              # " Hello " ← s 原封不动
s = s.strip()  # ✓ 想留住结果就赋回去
```

- **陷阱**：`s.strip()` / `s.replace(...)` / `s.upper()` 对 `s` 毫无作用，除非重新赋值。
- **对比**：列表方法如 `.append()` / `.sort()` **就地**修改并返回 `None`：`python nums = [3, 1, 2] nums = nums.sort() # ✗ nums 现在是 None！.sort() 返回 None nums.sort() # ✔ 就地排序；想要新列表用 sorted(nums)`

13 — `range` 不含终点；差一错误 (第3课)

```
list(range(1, 5))      # [1, 2, 3, 4]  ← 不包含 5
list(range(5))         # [0, 1, 2, 3, 4]
list(range(0, 10, 2)) # [0, 2, 4, 6, 8]
```

- **法则**：`range(a, b)` 是半开区间 `[a, b)`。

14 — 边遍历边修改列表 (第3课)

```
xs = [1, 2, 3, 4]
for x in xs:
    if x % 2 == 0:
        xs.remove(x)    # ✗ 跳元素 / 行为不可预测
# ✔ 遍历副本，或构建新列表：
xs = [x for x in xs if x % 2 != 0]
```

15 — 文件：`"w"` 会覆盖；游标会耗尽 (第8课)

```
open("data.csv", "w")    # ! 立即把文件清空
open("data.csv", "a")    # 追加
open("data.csv", "r")    # 读取 (默认)

with open("data.csv") as f:
    rows = f.readlines()  # 读了一遍
    again = f.readlines() # [] — 游标已在文件末尾
```

- 永远优先 `with open(...) as f:` —— 即使代码崩溃也会把文件关好。
- Windows 上配合 `csv` 模块，打开时带 `newline=""` 以免出现空行。

16 — 正则：`.` 匹配一切；用原始字符串 (第9课)

```
import re
re.search(r"\d+", "id 42")    # 用 r"..." 免得 \d 被当成 Python 转义
re.search(".", "a.b")         # 这个 "." 匹配到的是 "a"，不只是点！
re.search(r"\.", "a.b")       # 想要真正的点用 \.
```

- **法则**：模式永远写成原始字符串 `r"..."`。转义 `.` `^` `$` `*` `+` `?` `( )` `[ ]` `{ }` `|` `\`。
-

快速"先预测再运行"自测（盖住答案）

```
print(True + True)           # 2
print(3 == 3.0)              # True
print(0.1 + 0.2 == 0.3)      # False
print(5 == "5")              # False
print([1,2] == (1,2))        # False
print(bool("0"))             # True
print(7 // 2, -7 // 2)        # 3 -4
x = [1]; y = x; x.append(2); print(y)  # [1, 2]
```

八道题的为什么都能讲清，你就掌握了多数新手漏掉的根基。

Python 语法速查 —— 那些总记不住的写法

"那个来着怎么写?"专用表。按课程分组。

输入输出与类型 (第 1 课)

```
name = input("Name: ")          # 永远返回 str
age  = int(input("Age: "))       # 立即转换
print("Hi", name, age)          # 空格分隔
print(f"Hi {name}, age {age}")  # f-string (首选)
print(f"{score:.2f}")           # 两位小数
print(f"{count:,}")             # 千位分隔符: 1,234
print("no newline", end="")     # 去掉行尾的 \n
int("42"), float("3.14"), str(42), bool(0)  # 类型转换
type(x)                         # x 是什么类型?
```

f-string 格式化小抄

写法	示例	输出
:.2f	f"{3.14159:.2f}"	3.14
:,	f"{1234567:,}"	1,234,567
:>8	f"{'hi':>8}"	hi (右对齐)
:<8	f"{'hi':<8}"	hi (左对齐)
:^8	f"{'hi':^8}"	hi (居中)
:.1%	f"{0.873:.1%}"	87.3%

条件 (第3 课)

```
if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
else:
    grade = "F"

90 <= score <= 100          # 链式比较
grade = "pass" if score >= 60 else "fail"    # 三元表达式

match command:               # Python 3.10+
    case "start": ...
    case "stop" | "halt": ... # 多个值
    case _: ...              # 默认分支
```

循环 (第3 课)

```
for i in range(5): ...      # 0..4
for x in items: ...         # 每个元素
for i, x in enumerate(items): ... # 序号 + 元素
for a, b in zip(names, scores): ... # 两个列表并排走
while condition: ...
while True:                 # 输入校验循环
    x = input("...")
    if valid(x):
        break
# break / continue 控制流程
```

序列与切片 (第4 课)

```
xs = [1, 2, 3]; xs.append(4); xs[0]; xs[-1]    # 最后一个
xs[1:3]      # [2, 3]    (含起点, 不含终点)
xs[:2]       # 前两个
xs[::-1]     # 反转
t = (1, 2)   # 元组 (不可变)
d = {"name": "Ana", "gpa": 3.9} # 字典
d["name"]; d.get("missing", 0) # 访问; 带默认值的安全访问
d.keys(); d.values(); d.items()
s = {1, 2, 2, 3} # 集合 -> {1, 2, 3} (去重)
```

推导式 (第4/10 课)

```
[x*2 for x in xs]          # 列表
[x for x in xs if x > 0]    # 带过滤
{name: 0 for name in names} # 字典
{x % 3 for x in xs}        # 集合
(x*x for x in xs)          # 生成器 (惰性)
```

排序 (第4 课)

```
sorted(xs)                                # 新的有序列表
sorted(xs, reverse=True)
sorted(students, key=lambda s: s["gpa"])    # 按字段
sorted(students, key=lambda s: s["gpa"], reverse=True)
xs.sort()                                  # 就地排序 (返回 None!)
```

函数 (第5 课)

```
def avg(nums: list[float]) -> float:
    """Return the mean of nums."""        # 文档字符串
    return sum(nums) / len(nums)

def greet(name, greeting="Hello"): ...     # 默认参数 (不许用可变对象!)
def total(*args): ...                     # 任意个位置参数 -> 元组
def config(**kwargs): ...                 # 任意个关键字参数 -> 字典
func(*my_list)                            # 列表摊开成参数
func(**my_dict)                           # 字典摊开成关键字参数
```

异常 (第7 课)

```
try:
    n = int(value)
except ValueError:
    n = None
except (KeyError, IndexError) as e:
    print(e)
else:
    print("ok")                            # 只在没有异常时运行
finally:
    print("always")                        # 收尾, 总会运行

raise ValueError("score must be 1-5")
assert n > 0, "n must be positive"
```

文件与 CSV (第 8 课)

```
with open("f.txt") as f:          # 读取, 自动关闭
    text = f.read()
with open("f.txt", "w") as f:     # 写入 (会覆盖!)
    f.write("line\n")

import csv
with open("data.csv", newline="") as f:
    for row in csv.DictReader(f):  # row 是以表头为键的字典
        print(row["name"])

with open("out.csv", "w", newline="") as f:
    w = csv.DictWriter(f, fieldnames=["name", "gpa"])
    w.writeheader()
    w.writerow({"name": "Ana", "gpa": 3.9})
```

顺手的标准库 (第 8 课)

```
import statistics; statistics.mean(xs); statistics.median(xs); statistics.stdev(xs)
import random; random.choice(xs); random.randint(1, 6); random.shuffle(xs)
from datetime import date; date.today(); date(2026, 6, 26)
from pathlib import Path; Path("data.csv").exists()
import json; json.dumps(obj, indent=2); json.loads(text)
```

正则 (第 9 课)

```
import re
re.search(r"pattern", text)      # 任意位置的第一个匹配 (或 None)
re.fullmatch(r"\d{5}", zip)     # 整个字符串必须匹配
re.sub(r"\s+", " ", text)       # 折叠空白
m = re.search(r"(\w+)@(\w+)", s)
m.group(1)                       # 第一个捕获组
```

符号	含义
.	任意字符 (换行除外)
\d \w \s	数字 / 单词字符 / 空白
+ * ?	1+、0+、0 或 1
{m,n}	m 到 n 次
^ \$	开头 / 结尾
[...] [^...]	集合内 / 集合外
(...)	捕获组
a b	a 或 b

类 (第 10 课)

```
class Student:
    def __init__(self, name, gpa):
        self.name = name
        self._gpa = gpa
    def __str__(self):
        return f"{self.name} ({self._gpa})"
    @property
    def gpa(self):
        return self._gpa
    @gpa.setter
    def gpa(self, value):
        if not 0 <= value <= 4:
            raise ValueError("gpa out of range")
        self._gpa = value

s = Student("Ana", 3.9)
print(s)          # 使用 __str__
```

程序骨架 (每个脚本)

```
def main():
    ...

def helper():
    ...

if __name__ == "__main__":
    main()
```


术语表——大白话释义

写给懂研究、不懂代码的学习者。每个词条：一口气说清它是什么。

- **实参 (Argument)** —— 调用函数时递进去的值。 `int("5")` 里的 "5" 就是实参。
- **形参 (Parameter)** —— 函数定义里接收实参的命名占位。 `def f(x):` 里的 `x` 就是形参。
- **变量 (Variable)** —— 指向对象的一个名字 (标签)。不是盒子；是便利贴。
- **对象 (Object)** —— Python 存储的任何值：数字、字符串、列表、函数，甚至类型。"万物皆对象。"
- **类型 (Type)** —— 对象的种类：`int`、`float`、`str`、`bool`、`list`、`dict`、`set`、`tuple`、`NoneType`。
- **动态类型 (Dynamic typing)** —— 变量不锁定类型；有类型的是对象，不是名字。
- **`int` / `float`** —— 整数 / 小数。 `/` 永远得到浮点数；`//` 向下取整成整数。
- **`str`** —— 文本。不可变 (不能就地修改)。写在引号里。
- **`bool`** —— `True` / `False`。技术上是一种 `int` (`True == 1`)。
- **`None`** —— "没有值"。不 `return` 的函数返回的就是它。用 `is None` 检测。
- **`f-string`** —— `f"...{expr}..."` —— 带 `{}` 占位符、由 Python 填空的字符串。
- **相等 (`==`)** —— "值相同吗？"你通常想要的比较。
- **同一 (`is`)** —— "是内存里同一个对象吗？"只用于 `None` / `True` / `False`。
- **真值 / 假值 (Truthy / Falsy)** —— 放进 `if` 的值都算真，除非是 `0`、`""`、`[]`、`{}`、`set()` 或 `None`。
- **可变 / 不可变 (Mutable / Immutable)** —— 能否就地修改？`list` / `dict` / `set` 能；`int` / `str` / `tuple` 不能。
- **别名 (Aliasing)** —— 两个名字指向同一个可变对象；改一个，另一个也变。
- **切片 (Slicing)** —— 取子序列：`xs[start:stop:step]`，不含终点。
- **推导式 (Comprehension)** —— 用一行循环构建列表/字典/集合：`[x*2 for x in xs]`。
- **函数 (Function)** —— 可复用的具名代码块；接收输入，可选地 `return` 输出。
- **`return` vs `print`** —— `return` 把值交回调用者；`print` 只是往屏幕上显示文字。
- **作用域 (Scope)** —— 名字在哪里可见。Python 按 局部 → 闭包 → 全局 → 内置 (LEGB) 查找。
- **`*args` / `**kwargs`** —— 收集"任意个"位置参数 (元组) / 关键字参数 (字典)。
- **类型标注 (Type hint)** —— 对期望类型的可选注释 (`def f(x: int) -> str:`)。是文档，运行时不强制。
- **异常 (Exception)** —— 运行时抛出的错误 (如 `ValueError`)。用 `try/except` 处理。
- **`raise`** —— 主动抛出异常。**EAFP** —— "先斩后奏"：直接试，失败再接住。
- **`assert`** —— 开发者的自检，不成立就报错。不能用来校验用户输入 (可以被关闭)。
- **模块 (Module)** —— 可以 `import` 的 `.py` 文件。**库 / 包** —— 一捆模块 (用 `pip` 安装)。
- **标准库 (Standard library)** —— Python 自带的模块 (`csv`、`statistics`、`random`、`re`、`json`、`datetime`、`pathlib`)。
- **上下文管理器 (`with`)** —— 自动搭建并拆除资源的代码块 (比如自动关文件)。
- **CSV** —— 逗号分隔值；纯文本表格。`csv.DictReader` 把每行变成字典。
- **类 / 对象 (OOP)** —— 蓝图 (`class`) 和照它造出来的东西 (实例)。把数据和行为捆在一起。
- **`self`** —— 在类内部，指"眼下这一个实例"。
- **`__init__`** —— 创建实例时运行的初始化方法。**Dunder** = "双下划线"方法。

- `@property` —— 让方法看起来像属性；可以在读/写时做校验。
- 生成器 / `yield` —— 惰性地一次产生一个值；大数据省内存。只能遍历一次。
- `lambda` —— 极小的匿名函数，常用作排序的 `key=`。
- 正则表达式 (Regex) —— 匹配/提取文本的模式语言 (`re` 模块)。
- 回溯 (Traceback) —— 出错时 Python 打印的报告。先读最后一行。
- REPL —— 交互式 `>>>` 提示符，一次输入一行。

爬虫与抓包 —— 入门参考

写给想走向网络爬虫和数据包抓取的学习者。两者都不过是把课程里的基本功 拼装起来。完整可运行的示例在 `examples/networking/`。

首先，授权（读一遍，认真对待）

- 爬虫：遵守 `robots.txt` 和各网站的条款；用 `User-Agent` 表明身份；限速（每 1-2 秒一次请求）；有官方 API 就优先用 API。在自己的网站或沙盒（`quotes.toscrape.com`、`books.toscrape.com`）上练习。
- 抓包：只捕获你自己机器的流量，或你获得授权监控的网络。读自己抓的 `.pcap` 文件永远没问题；截取他人流量在很多地方是违法的。

网络爬虫工具箱

库	作用	标准库？
<code>requests</code>	通过 HTTP 抓取 URL (<code>requests.get(url).text</code>)	否 (<code>pip install requests</code>)
<code>urllib.parse</code>	拼接/拆解 URL —— <code>urljoin</code> 、 <code>urlparse</code>	是
<code>urllib.robotparser</code>	读 <code>robots.txt</code> ，用 <code>can_fetch()</code> 询问	是
<code>beautifulsoup4</code>	解析真实 HTML，找出链接/字段	否 (<code>pip install beautifulsoup4</code>)
<code>time</code>	请求之间 <code>sleep()</code> （礼貌）	是

```
import requests, time
from urllib.parse import urljoin, urlparse
from bs4 import BeautifulSoup

html = requests.get(url, headers={"User-Agent": "MyBot/1.0"}, timeout=5).text
soup = BeautifulSoup(html, "html.parser")
links = [urljoin(url, a["href"]) for a in soup.find_all("a", href=True)]
time.sleep(1)                                # 讲礼貌
```

爬虫的骨架（第 3、4 课）：一个 `deque` 前沿队列 + 一个 `visited` 集合。

```

from collections import deque
queue, visited = deque([root]), set()
while queue:
    u = queue.popleft()
    if u in visited: continue          # 集合防止无限循环
    visited.add(u)
    ... 抓取、提取链接、把同站的新链接入队 ...

```

礼貌爬虫清单： ☐ User-Agent ☐ robots.txt ☐ 超时 ☐ 请求间 sleep ☐ 只在一个站内 ☐ 限制页数 ☐ 处理 `requests.RequestException`。

下一步： `scrapy`（完整框架）、`playwright` / `selenium`（JS 渲染的页面）。

数据包抓取工具箱

库	作用	标准库？
<code>scapy</code>	捕获/构造/解析数据包，读写 <code>.pcap</code>	否（ <code>pip install scapy</code> ）
<code>struct</code>	从原始头部字节里解出数字	是
<code>socket</code>	底层网络原语	是

```

from scapy.all import rdpcap, sniff, IP, TCP
packets = rdpcap("capture.pcap")          # 离线：无需特殊权限
# packets = sniff(count=100)              # 实时：需 root + 授权
for p in packets:
    if IP in p and TCP in p:
        print(p[IP].src, "->", p[IP].dst, "port", p[TCP].dport)

```

数据包就是字节（第 2 课——bytes 与 str 陷阱）：

```

import struct
raw = bytes.fromhex("4500003c...")        # 20 字节 IPv4 头部
version = raw[0] >> 4                     # 4
hdr_len = (raw[0] & 0x0F) * 4             # 20 字节
total_len = struct.unpack("!H", raw[2:4])[0] # "!H" = 大端 uint16
protocol = raw[9]                         # 6=TCP 17=UDP 1=ICMP
src_ip = ".".join(str(b) for b in raw[12:16]) # "192.168.0.1"

```

`b"..."`（bytes）不是 `"..."`（str）：`raw[0]` 是一个 `int`，端口和标志位都是你常用十六进制读的整数。常见端口：80 HTTP · 443 HTTPS · 53 DNS · 22 SSH。

安全抓包清单： ☐ 只抓自己/授权网络的流量 ☐ 优先离线 `.pcap` 而非实时 ☐ 实时捕获需要 admin/root ☐ 过滤要窄（BPF：`sniff(filter="tcp port 80")`）。

下一步： `pyshark`（从 Python 调 Wireshark 的 `tshark`）、`dpkt`（快速 pcap），以及用 **Wireshark** 本体肉眼看抓包。

每一课带你走向哪里

第 1 课 字符串 → 拼/拆 URL · **第 2 课 bytes vs str** → **数据包与解码** · 第 3 课 循环 → 爬取/捕获循环 · 第 4 课 set+deque+Counter → visited 集合、前沿队列、计数 · 第 5 课 函数 → `fetch()/parse()` + 限速装饰器 · 第 6 课 递归 → 限深爬取、嵌套层 · 第 7 课 异常 → 超时与重试 · **第 8 课 requests/json/pathlib** → **抓取、调 API、保存、读 .pcap** · 第 9 课 正则 → bs4 → 提取字段 · 第 10 课 类/生成器 → 惰性流式的 `Crawler / Sniffer`。

简单（离线）版本和真实（`requests / scrapy`）作业都在 `examples/networking/`。

每课测验（含答案）

每份测验 5–6 题（最后一两题来自“更进一步”部分），约 8 分钟，课程结束时进行。每道题都对应本课的一个学习目标。**教师：**让学生在运行代码之前先**预测**输出——惊讶本身就是评估。答案在每课题目之后。

Session 1 — 运行 Python、变量与类型

1. `input("x: ")` 返回什么类型？永远如此吗？
2. `"5" + "3"` 的结果是什么？怎样得到 8？`int(3.9)` 又是什么？
3. `f"{0.873:.1%}"` 会输出什么？
4. 报错回溯里先读哪一行——它告诉你哪两件事？
5. `17 // 5` 和 `17 % 5` 各是什么——再说一个 `%` 的实际用途。

答案：1. `str`，永远如此。2. `"53"`；用 `int("5") + int("3")`；`int(3.9)` 是 3（截断，不是四舍五入）。3. `"87.3%"`。4. **最后一行**：异常的名字（哪类问题）和消息（出错的值或细节）。5. 3 和 2；`%` 给出余数——判断奇偶、把 ID 轮流分进 k 个组、把分钟拆成小时 + 分钟。

Session 2 — 动态类型陷阱

1. `a = [1,2]`；`b = [1,2]` —— `a == b` 吗？`a is b` 吗？为什么？
2. `True + True` 是什么？为什么？
3. `0.1 + 0.2 == 0.3` 是 True 还是 False？应该怎么比较？
4. `5 == "5"` 给出什么？`5 > "5"` 呢？
5. 下面哪些是假值：`"0"`、`""`、`[0]`、`[]`、`None`、`0.0`？
6. 为什么 `Decimal("0.1") + Decimal("0.2") == Decimal("0.3")` 成立而浮点版不成立——为什么必须从字符串构造 `Decimal`？

答案：1. `==` 为 True（值相同），`is` 为 False（内存中是不同对象）。

2. 2——`bool` 是 `int` 的子类（`True` 相当于 1）。3. False（二进制浮点舍入）；用 `math.isclose(0.1 + 0.2, 0.3)` 或先取整。4. False（不报错）；`5 > "5"` 抛 `TypeError`——Python 无法给 `int` 和 `str` 排大小。5. `""`、`[]`、`None`、`0.0` 是假值；`"0"` 和 `[0]` 是真值（非空）。6. `Decimal` 精确存储十进制数位，没有二进制舍入；从浮点数构造（`Decimal(0.1)`）会把浮点误差原样继承进来。
-

Session 3 — 控制流：条件与循环

1. 用链式比较重写 `if x >= 90 and x < 100:`。
2. `5 and 0` 是什么？`0 or "hi"` 是什么？为什么它们不是 `True/False`？
3. `list(range(1, 5))` 产生什么？

4. 一边遍历列表一边 `remove` 元素会出什么问题——怎么修？
5. `for` 循环的 `else` 子句什么时候运行？
6. `score = 95` 时，三个并排的 `if score >= 60 / 80 / 90:` 打印会发生什么——哪个关键字能修好它？

答案：1. `if 90 <= x < 100:`。2. `0` 和 `"hi"`——`and/or` 返回的是操作数，不是布尔值。3. `[1, 2, 3, 4]`（不含 5——差一错误）。4. 删除使后续下标前移，元素被跳过；改为构建新列表（`[x for x in xs if keep(x)]`）或遍历副本。

5. 仅当循环没有 `break` 时——“找遍了也没找到”的分支。6. 三个标签全部打印（每个 `if` 都是独立问题）；用 `elif` 串成一架梯子，只有第一个命中的分支运行。

Session 4 — 数据结构

1. 哪些是可变的：`list`、`tuple`、`dict`、`set`？
2. `a = [1, 2]; b = a; a.append(3)`——`b` 是什么？为什么？
- 3.（实操）一行字典推导式：把 `names` 里的每个名字映射到 `0`。
4. `xs.sort()` vs `sorted(xs)`——各返回什么？
5. 要对成千上万个 ID 做“这个 ID 出现过吗？”——选哪种结构，为什么？
6. `Counter(['a', 'b', 'a']).most_common(1)` 返回什么？

答案：1. `list`、`dict`、`set` 可变；`tuple` 不可变。2. `[1, 2, 3]`——`b` 是同一个列表的别名（赋值不复制）。3. `{n: 0 for n in names}`。4. `.sort()` 就地排序、返回 `None`；`sorted()` 返回新的有序列表。5. `set`——成员判断快，且天然不会有重复。6. `[('a', 2)]`——（值，次数）对的列表，次数最多的在前。

Session 5 — 函数、作用域与复用

1. `return` 和 `print` 的区别是什么？
2. `def f(x, items=[])` 为什么危险？怎么修？
3. 没有 `return` 语句的函数返回什么？类型标注在运行时强制吗？
4. 顶层 `count = 0`，函数内部写 `count = count + 1`——什么错误，为什么？
5. 一句话：装饰器做什么？`def f` 上方的 `@d` 是什么的语法糖？

答案：1. `return` 把值交给调用者；`print` 只是显示。2. 默认列表在 `def` 时只创建一次，跨调用一直存在；用 `items=None` 然后在函数内创建。3. `None`；不强制——标注是文档（`mypy` 可选检查）。4. `UnboundLocalError`——赋值让 `count` 成为函数局部变量，于是右侧读到一个尚不存在的局部名。5. 它给函数包上额外行为；`@d` 就是 `f = d(f)`。

Session 6 — 递归与递归思维

1. 每个递归函数必须有哪两部分？
2. 基例永远到不了会出什么错？Python 为什么不一直跑下去？

3. 递归为什么天然适合嵌套数据（列表套列表、嵌套 JSON）？
4. 递归步算了 `n * f(n - 1)` 但前面没写 `return`——调用会得到什么？
5. 朴素递归 `fib(35)` 要好几秒而 `@functools.cache` 让它瞬间完成——缓存改变了什么？

答案：1. **基例**（停下）和**递归步**（对更小的输入调用自己、朝基例走）。

2. `RecursionError`——Python 没有尾调用优化，每个未完成的调用都占一个栈帧，到上限（约 1000）为止。
3. 数据以**自身定义**（列表可以含列表），以自身定义的函数与数据的形状互为镜像，能到达每一层。
4. `None`——算出的值被丢弃；函数走到结尾就返回了。
5. 每个不同输入只算一次并被记住，指数级的重复子调用坍缩成约 35 次查表。

Session 7 — 异常与防御式编程

1. `int("N/A")` 抛哪种异常？把 `int(value)` 包一下，失败时返回 `None`。
2. 裸 `except:` 为什么危险？
3. 什么时候用 `raise`，什么时候用 `assert`？
4. 说出两种校验风格——并给出把 `value` 转成 `int` 的 EAFP 写法。
5. 为什么 `except ValueError:` 必须放在 `except Exception:` 前面？

答案：1. `ValueError`；`try: return int(value) / except (ValueError, TypeError): return None`。

2. 它捕获一切（连 Ctrl+C 和你自己的手滑都算），会把 bug 藏起来。
3. 校验真实/不可信输入用 `raise`；开发者的自检用 `assert`（`python -O` 可关闭）。
4. LBYL（“三思而后行”：`if value.isdigit():`）vs EAFP（“先斩后奏”）：`try: n = int(value) / except ValueError: ...`。
5. 处理器自上而下匹配，`Exception` 也能接住 `ValueError`——放前面会把一切吞掉，具体的处理器永远轮不到。

Session 8 — 文件、库与研究数据

1. 用 `"w"` 模式打开文件对已有内容做什么？为什么要用 `with open(...)`？
2. `csv.DictReader` 读出的每一行是什么类型？
3. 从文件读出的 CSV 值是什么类型——数字必须做什么？
4. 对打开的文件对象做第二次循环什么都读不到——为什么？怎么修？
5. `strptime` 和 `strftime` 各做什么——分析脚本里为什么要给 `random` 设种子？
6. 脚本以 `python3 report.py survey.csv` 运行——`sys.argv` 是什么？下标 0 是什么？

答案：1. **立即**清空文件；`with` 保证即使代码崩溃也会自动关闭。

2. 以表头行 为键的 `dict`。

3. 字符串；用 `int()/float()` 转换。

4. 文件游标读完一遍就停在 末尾——重新打开（或先读进列表）。

5. `strptime` 解析文本 → `datetime`；`strftime` 把 `datetime` 格式化 → 文本；设种子让“随机”抽样可复现，方法部分写得出来。

6. 一个普通的字符串列表 `['report.py', 'survey.csv']`；下标 0 永远是脚本自己的 名字，真正的参数从 1 开始——先查 `len(sys.argv)`，不对就 `sys.exit("usage...")`。

Session 9 — 正则表达式与文本清洗

1. 正则里 `.` 匹配什么？怎样匹配一个真正的点？
2. 为什么正则模式要写成原始字符串 `r"..."`？
3. 没匹配到时 `re.search` 返回什么？调 `.group()` 之前必须做什么？
4. `re.search` vs `re.fullmatch` ——哪个用于校验，为什么？
5. `re.IGNORECASE` 改变什么？`re.VERBOSE` 是干什么的？

答案：1. 任意字符（换行除外）；真正的点用 `\.`。2. 免得反斜杠被当成 Python 字符串转义。3. 返回 `None`；先 `if m:` 再 `m.group()`，否则会撞上 `AttributeError`。4. `fullmatch` ——它锁住两端，整个字符串必须符合模式；`search` 会接受垃圾中间的一段看似合法的片段。5. `IGNORECASE` 忽略大小写；`VERBOSE` 允许模式跨行、带注释，让同事真的能审阅它。

Session 10 — 模块、面向对象与 Python 惯用法

1. 类里的 `self` 是什么？
2. 把一个生成器遍历两次会怎样？
3. 模块的 `if __name__ == "__main__":` 块为什么在被 `import` 时不运行？
4. `class Course: students = []` ——两个 `Course` 对象各自招生时会出什么问题？怎么修？
5. `dataclass` 里为什么写 `courses: list = field(default_factory=list)` 而不是 `= []`？

答案：1. 当前实例（“眼下这一个对象”）。2. 第二遍是空的——生成器遍历一次就耗尽。3. 被导入时 `__name__` 是模块名，不是 `"__main__"`，块被跳过。

4. `students` 是所有实例共享的类变量，两门课看到的是同一份合并名单；在 `__init__` 里给每个实例自己的：`self.students = []`。5. 裸 `[]` 默认值只创建一次、被所有实例共享——第 5 课的可变默认值 bug 换了件类的外衣；`default_factory` 为每个实例现造一个新列表。